数字生命：一个可测量的生命定义

实验卷（最早期的实验记录）

> 何淇 · 与 DeepSeek（AI）协作

> 数字生命系列 · 实验卷（最早期的实验记录）

> 日期：2026-11-03

> 本卷为系列合集的分卷版；完整版见系列总合集。各篇单独 DOI 保留，以总合集与各篇发布记录为准。

卷首简介

本卷收录系列里的实验部分（第六~第二十二篇，共 17 篇：两篇测量 + 十五个实验）。

需要说明一句：**这些实验是我们最早讨论定义时期做的。** 后来理论、术语和结论口径有演进，部分实验的引言和定位反映的是当时阶段的表述；本卷保留原样，作为那个阶段的记录。后续基本不再更新本卷。

阅读建议：先读理论卷（至少第零、一、二、三篇与附录 B 术语表），再读本卷；遇到与最新术语不一致的表述，以理论卷和总合集为准。

目录

第三部分 测量与实验

- 第六篇　容量不能替代数据的更新
- 第七篇　替换与累积
- 第八篇　结构换算力：混合对齐语料与样本效率
- 第九篇　颜色作为第一块积木：感知基元与语言对齐的最小实验
- 第十篇　生长与记忆：自指更新与维持的最小实验
- 第十一篇　语义残差的测量：文本极限与接地补齐

- 第十二篇　分层是对动态的补偿：一个自适应深度的玩具实验

- 第十三篇　厚度与随机性：冻结模型在哪里最不稳定

- 第十四篇　压缩的形状：叙事与归档

- 第十五篇　最小切片与时间换维度：能力边界的两个实验

- 第十六篇　残差底与组合律：两个大模型结构实验

- 第十七篇　按计算厚度分配算力：预算与动员

- 第十八篇　层厚放在哪：异质层实验

- 第十九篇　RLHF 压平人类分歧

- 第二十篇　空间、组合与厚度：三个资源的交换实验

- 第二十一篇　层的路径与冗余：GPT-2 上的层扰动实验

- 第二十二篇　语言结构作为自变量：GPT-2 上的三组实验

第六篇 · 容量不能替代数据的更新

> 单独 DOI：https://doi.org/10.5281/zenodo.23503049

语言升维下的失配测量（**scaling vs** 持续学习：一个弱结论）

> 论文系列 · 第二章

> **DOI: https://doi.org/10.5281/zenodo.23503050**

> 版本：v2（2026-10-22 增补「维度与贴合」一节，见第四·五节）

> 第一章：《现在的 AI 为什么不是生命》（DOI: https://doi.org/10.5281/zenodo.23488543）

摘要

第一章提出：生命是「被维持的解压」，AI 是「一次性的解压」，并预测冻结模型会因语言升维出现**相对失配**。

本章给出测量，并**在过程中自我修正了两次方法错误**：

（1）先用「旧模型 vs 纯新模型」测，发现正反向幅度接近（+0.108 vs +0.128）⇒ 对称差异，测不出"落后"；

（2）改用**累积参照系**（旧模型 vs 懂全部年份的模型），失配随截止年变晚单调下降

（+1.026 → +0.721 → +0.322 bits/token），但对照也显著为正 ⇒ 混入「数据量」效应；

（3）最后用**等数据预算对照**（三个模型训练数据量相同，只差数据新旧），分离出概念新鲜度的独立效应：

只用最近数据的模型在 2024 测试集上比只用最早数据的模型好 **0.405 bits/token**（同样数据量），

而混合新旧数据的模型在旧测试集上几乎不损失（13.655 vs 13.659）。

并进一步做了**容量补偿实验**：把模型容量从 2 阶提到 6 阶（带插值 n-gram），

旧数据与混合数据的差距稳定在 **+0.86~0.94 bits/token**，未见缩小

⇒ 在本设置下**未见容量补偿效应**（弱结论：不构成对 scaling 的否定，需 transformer 复核）。

结论：**语言是累积升维的 —— 加新概念不丢旧概念。因此正确的更新是「累积/混合」而不是「替换」：

替换会遗忘（只用新数据时旧语料 13.800 vs 13.659），冻结则停在训练时刻。

持续更新的最优策略 = 混合新数据、保留旧能力。** 全部脚本可复现。

一、问题

```

若语言随时间增加新的「维度」（新概念、新用法）

⇒ 冻结在训练时刻的模型，对这些新维度没有行为谱

问题：

① 这个差距有多大？

② 它来自「概念新旧」还是「数据多少」？

③ 加模型容量（参数）能不能补偿？

④ 正确的更新方式是什么：替换还是累积？
```

```

## 二、数据与方法

**数据**：arXiv cs.AI，2018 / 2020 / 2022 / 2024 各 3000 篇（标题+摘要），共 12000 篇。

抓取：`502_fetch_arxiv.py`。

**两次自我修正（方法）**

```

初始（504）：旧模型 vs 纯新模型 —— 正反向对称 ⇒ 错误，测不出落后

修正一（505）：累积参照系 —— 方向对了，但有数据量混淆

修正二（506）：等数据预算 —— 三个模型数据量相同，只差新旧

```

**容量实验（507/508）**

```

固定数据量 N=1800 篇；把 n-gram 阶数 2→6 作为容量代理

508 用 Jelinek-Mercer 插值，使高阶更稳

比较 OLD / MIX / REC 三组在 2024 留出集上的 bits/token

```

## 三、结果

**3.1** 升维速率的词层面代理（`**501_vl_newterm.py**`）

2018 vs 2024（6 年）：新词占比 **7.30%**（≈ **1.2%/年**），新概念 7787 个。

新概念 Top：large language, llms, prompting, rag, generative ai,

vision-language models, retrieval-augmented, gpt-4o 等。

**3.2** 等数据预算对照（核心结果，`**506**`）
```

bits/token（三者数据量完全相同）：

| 模型（数据量相同） | 2024 测试集 | 2018 测试集 |

|---|---|---|

| A 只用最早年份（2018） | 14.026 | 13.659 |

| B 混合全部年份 | 13.742 | 13.655 |

| C 只用最近年份（2024） | **13.621** | 13.800 |

```

① C 比 A 在 2024 测试集好 0.405 bits/token（数据量相同）

⇒ 概念新鲜度有**独立价值**

② B（混合）新测试 13.742，旧测试 13.655 ≈ A 的 13.659

⇒ 混入新数据：新数据变好，旧能力不丢 ⇒ **更新有净收益**

③ C 在旧测试集变差（13.800）

⇒ 只用新数据会遗忘旧概念 ⇒ **替换有代价**

```

3.3 容量补偿实验（**`508`**，带插值）

```

最大阶 OLD MIX REC OLD−MIX

2 10.712 9.774 9.505 +0.938

3 10.946 10.059 9.808 +0.888

4 11.011 10.143 9.901 +0.868

5 11.029 10.169 9.931 +0.860

6 11.035 10.178 9.942 +0.857

```

```

① OLD−MIX 差距几乎不变（0.938 → 0.857）

⇒ 加大容量**没有补偿**「数据太旧」

⇒ 弱结论：本实验设置下未见容量补偿（不构成对 scaling 的否定）

② REC 在新语料上最好（9.94），但旧语料会变差（见 3.2）

```

3.4 累积参照系（**`505`**，作为方法展示）

| 截止年 | M（新测试） | M_ctl（旧测试） |

|---|---|---|

| 2018 | +1.026 | +0.808 |

| 2020 | +0.721 | +0.459 |

| 2022 | +0.322 | +0.151 |

M 单调下降（知道得越多，落后越少）；

但 M_ctl 也为正 ⇒ 数据量混淆 ⇒ 必须用 3.2 的等预算对照才能分离。

四、结论

```

① 语言升维真实存在：6 年跨度，概念新鲜度的独立价值约 0.4 bits/token

② 但很慢（词层面约 1.2%/年）；冻结模型的失配是缓慢累积的，不是断崖

③ 加容量补不了数据旧（**弱成立**，见下方三档）

④ 语言是**累积**升维的：加新概念不丢旧概念

⇒ 正确的更新是「累积/混合」，而不是「替换」

⇒ 冻结 = 一次性解压；替换 = 单维度解压；累积 = 维持解压
```

—— 最后这一条，正好对应第一章「生命 = 被维持的解压」的条件

```

**结论强度三档（本文只主张第二档）：**

```

强：参数不能替代新数据 —— 需要 transformer 级证据，本文未达到

弱：在本实验设置下未见容量补偿 —— 本文主张的就是这一档

零：scaling 无效 —— 本文没有任何证据支持，不主张

```

「弱结论」代表什么（不是文字游戏，要写清楚）

```

它代表：数据更新与模型扩容是两个**不同的杠杆**，在本设置下不能互相替代。

· 不代表「scaling 无效」——scaling 在固定数据分布内仍然有效

· 不代表「参数越多越坏」——只表示多出来的参数换不来对陈旧数据的补救

· 代表：失配是「数据面」的问题，不是「参数面」的问题

⇒ 若语言持续累积升维，正确对策是**更新数据（混合）**，

而不是只把模型做大

现实含义：

只扩模型不更新数据的策略，会在新语料上持续落后约 0.4~0.9 bits/token

科学含义：

「持续学习」不能简化为「把参数加够」——

维度（概念）的增长需要新数据来承载，参数只是接收器。

边界（再次声明）：

以上是 n-gram 容量代理下的**弱结论**；transformer 级验证前，

不能推广为「越大越没用」。

```

## 四·五、维度与贴合：一个补充讨论 [推测]

> 本节为 v2 新增，是**推测性讨论**，不改变前文任何实证结论；其中「有效维度」尚未测量。

一个经验事实：**某些小模型在特定领域强于更大的模型。** 如果能力只由「总维度（规模）」决定，这不该发生。这说明需要区分两个量：

```

覆盖（总维度）：模型能表示的结构总量 —— 由规模/容量决定

贴合（有效维度）：这些维度中，真正对准目标结构的部分 —— 由数据与训练分布决定

```

由此可以同时解释三个现象：

1. **升维仍持续带来能力提升**：覆盖在变广，更多结构类「够得着」⇒ 平滑上涨。

2. **能力会「突然出现」**：具体能力对应「某个结构是否被覆盖到」，是阈值式的（0/1），所以表现为跳变，而非渐变。

3. **小模型在窄领域反超**：它的覆盖小，但可以把维度集中对准该领域的少数关键结构 ⇒ 局部贴合度更高。

回到本文的结论：**容量是接收器，数据决定方向。** 加大容量能扩大覆盖，但不能把维度「对准」新数据带来的新结构；这正是「容量不能替代数据更新」的另一种说法——数据决定有效维度，容量只决定这些维度能展开到多大。

可检验的预测（尚未做）：若能用表征方法测量「有效维度」，则当某任务所需结构维度 k 大于模型的覆盖维度时，该能力不应出现；一旦覆盖维度越过 k，能力应阈值式出现。这需要一个可操作的有效维度度量——这是下一章留下的问题。

## 五、限制（必须与结论一起读）

```

· 仅一个领域（cs.AI）、一个来源（arXiv 摘要）

· n-gram 是极简语言模型：测的是词级概念失配，不代表大模型语义/推理层面的失配

· 容量代理 = n-gram 阶数，不是真正的参数量；高阶 CE 仍偏大（插值 λ 固定）

⇒ 容量结论只能算初步证据，需 transformer 小模型复核

· 等预算控制的是训练文档数（非 token 数）；采样只用一个随机种子

```

## 六、可复现

```

python 502_fetch_arxiv.py cs.AI 2018 2024 # 及 2020/2022

python 501_vl_newterm.py <语料根> 6 # 新概念率

python 504_series.py <2018> <2020> <2022> <2024> # 对称性检验（展示错误）

python 505_cumulative_mismatch.py <四年目录> # 累积参照系

python 506_equal_budget.py <四年目录> # 等预算对照（核心）

python 507_capacity_vs_data.py <四年目录> # 容量初探（无回退）

python 508_backoff_capacity.py <四年目录> # 容量补偿（带插值）

```

**作者：何淇（独立研究者）；AI 协作：DeepSeek**

## 第七篇 · 替换与累积

> 单独 DOI：https://doi.org/10.5281/zenodo.23503559

> 论文系列 · 第三章

> 第一章：《现在的 AI 为什么不是生命》 DOI: 10.5281/zenodo.23488543

> 第二章：《容量不能替代数据的更新》 DOI: 10.5281/zenodo.23503050

## 摘要
```

当前 AI 领域有三个看似独立的热点争议：**模型崩塌**（用模型生成的数据训练导致退化）、

压缩陷阱（追求压缩可能牺牲真相）、**scaling vs 持续学习**（堆参数还是持续更新）。

本文指出：这三者是**同一个问题的三个侧面** ——

语言是累积升维的；替换（而非累积）会切断维度生长。

（1）模型崩塌 = 用模型生成的数据**替换**人类数据 ⇒ 累积链被切断 ⇒ 维度退化；

（2）压缩陷阱 = 压缩的对象是**有损的人类语言**，最省编码不等于最符合世界；

（3）scaling vs 持续学习 = 容量只能接收维度，不能生成维度；第二章实测表明容量无法补偿数据陈旧。

由此得到一个可检验推论：**人类数据与模型数据的混合比例不应完全替换**

（纯合成数据 = 替换 = 崩塌；混合 = 累积 = 维持）。本文给出该推论的测量方案。

一、三个热点，一个结构

```

① Model collapse（Shumailov et al. 2023）

合成数据递归训练 ⇒ 分布尾部消失、多样性退化

现有解释：误差累积、分布估计漂移

② Compression trap（Rae 2024 等）

追求压缩 ⇒ 模型可能选"更简单但更不真"的理论

现有解释：目标函数与真理的偏离

③ Scaling vs continual learning

规模是否还有效？何时必须持续更新？

现有争论：benchmark 上升 vs 分布漂移风险

```

它们的共同点被忽略了：三者都发生在「语言是累积的」这一事实上。

二、统一框架：累积与替换

第一章的定义：生命 = 被维持的解压；AI = 一次性的解压。

第二章的实测：语言在累积升维（约 1.2%/年），且——

等数据预算下，混合新旧数据的更新有净收益且不遗忘；替换会遗忘。

把这一点推广到三个热点：

```

① 模型崩塌 ≈「替换」的极端情形

人类数据被合成数据替换 ⇒ 维度不再从真实世界增长

⇒ 分布尾部（= 稀有维度）首先消失 ⇒ 崩塌

⇒ 预测：混合比例越高（人类:合成 > 0），崩塌越慢；

纯合成 = 替换 = 最快崩塌

② 压缩陷阱 ≈「有损源」的必然结果

语言是世界的**有损投影**（第一章已述）

⇒ 在语言上最优的压缩，无法保证对世界为真

⇒ 压缩目标本身不携带"求真"约束

③ scaling vs 持续学习 ≈「接收器 vs 生成器」

参数是维度的**接收器**；概念维度的**生成**来自数据

⇒ 容量补不了数据陈旧（第二章弱结论）

⇒ 不是"哪个更重要"，而是"两者缺一不可"

```

三、可检验推论

```
```

推论 A（模型崩塌的混合比例实验）：

用同一基座模型，分别以 0% / 25% / 50% / 100% 的人类数据替换为合成数据

递归训练 k 代，测量：

· 生成分布的尾部覆盖率（稀有词/新概念保留率）

· 与真实人类语料的交叉熵

⇒ 预测：纯替换组崩塌最快；混合组按人类比例延缓

推论 B（压缩陷阱的语言层检验）：

在"只压语言"与"语言+可干预测量"两种设置下训练

⇒ 预测：只有后者能在新概念出现时保持对世界的校准

推论 C（scaling vs 累积）：

等算力预算下：加大模型 vs 混入新数据

⇒ 第二章已给初步答案：容量不能替代数据（弱结论）

```

## 四、为什么这个统一值得说

```

· 三个热点各自有文献，但没人把它们归到同一个结构（累积 vs 替换）

· 它给出可操作的实践含义：合成数据必须与人类数据**混合**，不能替换

· 它和第一、二章形成闭环：

第一章（定义：被维持的解压）

第二章（测量：累积最优，容量补不了数据）

第三章（应用：三个争议是同一件事）

```
```

五、限制

```

· 本文是立场/框架文章，未包含模型崩塌的原始实验

· "混合比例"推论需实验验证（本文只给方案）

· 三个热点的文献各自庞大，本文只取与"累积/替换"直接相关的部分

```

六、结论

```

模型崩塌、压缩陷阱、scaling vs 持续学习不是三个问题，

而是「语言累积升维、替换切断累积」的三种表现。

⇒ 正确的做法不是"更多合成数据"或"更大模型"，

而是**保持人类数据与真实世界的累积通道**。

```

作者：何淇（独立研究者）；AI 协作：DeepSeek

第八篇 · 结构换算力：混合对齐语料与样本效率

> 单独 DOI：https://doi.org/10.5281/zenodo.23542969

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 实验篇 2（内部编号第 9 篇）

> 日期：2026-10-22

> DOI: https://doi.org/10.5281/zenodo.23542969

摘要

当缩放（scaling）的成本越来越高，效率竞赛还有另一条少被系统化讨论的路：**不改模型、不改数据量，只改语料的结构。** 本文提出并检验一个工程假设：模型的样本效率不只取决于语料的数量，更取决于语料的结构。人类儿童同时通过语言与和世界的交互学习；当前训练语料则大多按通道切分——纯文本、纯图像、纯结构化数据各学各的，通道之间没有对齐。

我们用一个合成世界做受控实验。同一批事件，构造三种语料：第一，纯文字，只有句子；第二，混合对齐，句子加世界状态标签（物体大小、动作结果），插在对齐位置；第三，混合打乱，标签随机配到别的事件，作为控制。用插值 n-gram 模型在三种语料上训练，再**只给文字**做探针测试。

结果：混合对齐语料在 200 个事件时就学到「动词到大小」的接地（准确率 1.000）和「文字加大小到结果」的规则（1.000）；纯文字语料到 3200 个事件仍然只能瞎猜（0.500）或停在多数类基线（0.765）。打乱对齐的控制组掉回随机附近（0.333 到 0.722），说明收益来自**对齐**，不是来自标签变多。同时，语言建模的 bits/token 没有变差（0.79 到 0.81）。

结论：**在语料总量不变的前提下，把多通道信息对齐地混进同一份语料，可以用更少的样本学到更多结构，而且不牺牲语言能力。** 本文定位于**一个设想加一次原理验证**：合成世界上的证据只用来把话说清楚；真实多模态数据上的验证，留给愿意做的人。

更直白地说：更少的数据，更多的结构；在本文设置下，这还意味着更高的任务准确率——200 个事件即达到 1.000，而纯文字语料即使到 3200 个事件仍停在基线（0.500 与 0.765）。这里的"更强"指任务层面的结构可达性，不指通用能力上限。

一、问题

当前训练语料按通道切分：

第一类是文本语料，即语言对世界的投影；第二类是图像与视频语料，即视觉对世界的投影；第三类是结构化数据（数据库、仿真状态、执行结果），即逻辑对象与因果的投影。

每个通道都是世界函数 S 的一个投影 P_i(S)；单独一个投影是不完整的。模型要恢复更高维的结构，需要同一结构被多个通道同时记录，并且对齐：同一个对象、同一时刻、同一个事件。

人类儿童的学习方式正是如此：语言、视觉、触觉、与他人的交互同时发生，词与对象、动作与后果当场对上。本文问一个可测的问题：在总数据量相同的前提下，对齐的多通道语料能否比单通道语料学到更多结构？

二、受控实验设计

2.1 合成世界

物体有三个属性：形状属于「球、方块」，颜色属于「红、蓝」，大小属于「小、大」。动词编码大小：表示小的动词有「拿起、托起、捧起」；表示大的动词有「扛起、抬起、抱起」。事件句子是「他＋动词＋了＋颜色＋的＋形状」。结果规则是：成功当且仅当形状是球，或者大小是小。关键设定：文字里只有形状，没有「大小」这个词；结果也不在文字里。

2.2 三种语料（同一批事件）

第一种，纯文字：只有事件句。第二种，混合对齐：事件句，且在动词后插入大小标签、句末插入结果标签。第三种，混合打乱：同第二种，但标签来自随机另一个事件，作为控制组。

2.3 探针（测试时只输入文字 **token**）

第一个探针测接地：输入「他、动词」，输出大小，瞎猜基线为 0.50。第二个探针测结构：输入文字加真实大小标签，输出结果，多数类基线为 0.75。第三个探针测语言：在纯文字留出集上算 bits/token，越低越好。

2.4 模型与预算

插值 n-gram，阶数 6，插值系数 0.5，纯 Python 实现。事件数取 200、800、3200 三档，各跑 3 个随机种子取平均。

三、结果

事件数为 200 时：纯文字语料，接地 0.500，结构 0.765，语言 bpt 0.81；混合对齐语料，接地 1.000，结构 1.000，语言 bpt 0.81；混合打乱语料，接地 0.333，结构 0.807，语言 bpt 0.83。

事件数为 800 时：纯文字语料，接地 0.500，结构 0.765，语言 bpt 0.79；混合对齐语料，接地 1.000，结构 1.000，语言 bpt 0.79；混合打乱语料，接地 0.333，结构 0.759，语言 bpt 0.79。

事件数为 3200 时：纯文字语料，接地 0.500，结构 0.765，语言 bpt 0.79；混合对齐语料，接地 1.000，结构 1.000，语言 bpt 0.79；混合打乱语料，接地 0.722，结构 0.776，语言 bpt 0.79。

三个观察：

第一，样本效率。混合对齐在 200 个事件就学会了接地与规则；纯文字到 3200 个事件仍然没有这两个概念。

第二，对齐是关键。打乱标签后两个探针掉回随机或基线附近，说明收益来自通道与事件的对齐，不是来自多了几个 token。

第三，语言没有受损。三种语料的语言 bpt 在同一水平（0.79 到 0.81），混入通道标签没有牺牲语言建模。

四、为什么有效（机制讨论）

第一，一个样本同时监督多个通道。同一事件里，句子、大小、结果同时出现，每份数据的信息产出更高。

第二，相互消歧。文字里没有大小，但动词与大小相关；通道把这个相关性钉死，模型于是学会从文字推断世界状态，这就是接地。

第三，结构可迁移。从「动词到大小到结果」学到的规则，可以在新组合上直接使用（探针使用的是留出事件的文字）。

一句话：混合对齐语料把语言监督和世界监督合并成同一份样本，而不是各训练一遍。

四·五、课程顺序：按结构依赖的拓扑序学

还有一个隐含的机制，本文的实验没有直接测，但值得单独说：对齐解决的是"通道"问题，**顺序**解决的是"难度"问题。两者叠在一起，才有"像小孩一样学"的效果。

人类儿童的发育顺序不是随机的：先建立对象（哪些东西是同一个东西），再建立关系（谁在哪、什么连着什么），再建立动作与后果（做什么会怎样），最后才是用语言去指这些东西。这个顺序不是教育者的偏好，而是结构本身的依赖顺序——上层结构依赖下层结构，反过来不成立。

因此一个自然的预测是：同样的对齐语料，如果按依赖关系排序喂给模型（课程），会比随机顺序学得更快。理由有三条。第一，每一步的有效假设空间小得多，模型只需要学当前这一层。第二，每一层都有对齐通道直接监督，不靠语言间接推断。第三，上层结构建立在下层已验证的结构上，不会因为底层错了而推倒重来。

换句话说：**对齐让样本变"厚"，课程让学习变"省"。** 本文的合成实验只验证了前者；后者是一个可测但未测的预测，列在第七节的验证清单里。

五、局限（必须与结论一起读）

第一，合成世界：规则简单、无噪声、通道是标签而不是真实视觉或动作。

第二，模型是 n-gram：只验证原理，不代表 transformer 或真实多模态数据。

第三，对齐由实验者构造：真实数据中的对齐（时间、对象、指代）本身是难点。

第四，没有测分布外泛化与长期使用中的漂移。

六、可复现

脚本为 700_mixed_corpus_experiment.py（初版）、701_mixed_corpus_v2.py、702_mixed_corpus_v3.py、703_mixed_corpus_v4.py（最终版）。运行方式：python 703_mixed_corpus_v4.py。依赖：Python 3 标准库，无第三方。

七、给后来者的验证清单

本文到此为止，不做更多实验。以下问题留给愿意验证的人。

第一，真实数据：用 Ego4D、AI2-THOR、Open X-Embodiment 等已对齐数据，做同样的「混合对齐对单通道」对照，看增益是否还在。

第二，规模：从 n-gram 换到小 transformer，看样本效率增益随规模的变化。

第三，对齐质量：人为引入错配率（0%、10%、30%、50%），画出「错配率到增益」的曲线。

第四，通道独立性：加入冗余通道（同一信息的另一种编码），检验增益是否消失（预期：冗余通道不带来增益）。

第五，结构类型：换不同类型的规则（因果、层级、时序），看哪类结构最依赖外部对齐通道。

第六，理论：给出一个粗略的样本复杂度表达式，把对齐带来的互信息与样本效率联系起来。

第七，课程顺序：同一套对齐语料，按结构依赖的顺序喂（对象、关系、动作与后果、语言）与随机顺序对照，比较各自的样本效率。这一条检验的是第四节·五的预测。

如果有人在真实数据上重复出方向一致的增益，这个设想才算立住；如果重复不出，本文就是错的。两种结果都有价值。

附：推测与边界

「真实多模态数据上同样有效」是 [推测]，本文未测。「对齐质量决定收益大小」是方向性判断，需要真实数据上的对照。本文不主张语料结构可以完全替代数据量，只主张同量下结构更好、样本效率更高。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第九篇 · 颜色作为第一块积木：感知基元与语言对齐的最小实验

单独 DOI：https://doi.org/10.5281/zenodo.23551953

何淇 · 与 DeepSeek 协作

数字生命系列 · 实验篇 3（内部编号第 10 篇）

日期：2026-10-22

摘要

如果要把视频与音频变成可训练的结构化语料，第一步不是训练更大的模型，而是先回答一个更小的问题：**一个感知基元，能不能只靠"语言 + 感知通道"的对齐被学到？** 本文选颜色作为最小测试场——它是唯一同时具备物理定义（波长/色相）、感知分解（色相、饱和度、明度）与简单语言标签（基本颜色词）的感知基元。

我们用 12 个基本颜色词的 HSV 样本构造三种语料：纯文字（只有颜色词与关系句）、对齐（颜色词与 HSV 分箱坐标双向配对）、打乱（坐标随机配到别的词，作为控制组）。用插值 n-gram 在三种语料上训练，再做三个探针：词到坐标（P1）、坐标到词（P2）、关系判断（P3，暖或冷）。

结果三种语料一致：关系判断都能学会（约 94% 到 100%）；但坐标到词只有对齐语料学得会（约 86% 到 91%），纯文字为 0，打乱组掉回随机水平（约 8% 到 11%）；词到坐标在对齐语料下为 31% 到 44%，受分箱噪声限制，不是满分。结论是：**颜色可以作为感知基元被对齐语料接地；纯文字只能学关系，不能接地；收益来自对齐本身，而不是多给了 token。**

本文是一次原理验证，世界与模型都是合成的；它要回答的不是"AI 能不能看懂颜色"，而是"感知基元接地这件事，最小可测的形式是什么样"。这个问题一旦成立，视频与音频按事件切分后走同一条路才有依据。第四节之后我们补一节说明它的意义：如果感知基元能被对齐接地，那么"模型自己挖视频、自己造对齐语料、再让自己贴合文字维度"这条自训练路径，就有了最底层的第一个例子。

一、问题

视频与音频是下一座数据矿。但"把视频变成语料"不是工程问题，而是定义问题：先要定义一个单位（一件事），再让画面、声音、文字在同一条时间轴上对齐。

在定义那个大单位之前，值得先验证一个更小的单位：**感知基元**。问题可以写成一句话：给一个感知基元配上它的语言标签，模型能不能学到"词与感知坐标"之间的映射？

颜色是合适的起点，原因有三条。第一，它可以被精确定义：色相是一个 0 到 360 度的连续量，可以离散成有限个区间。第二，它可以被分解：色相、饱和度、明度三个维度，正好是一个小的、可枚举的结构。第三，它有简单的语言标签：基本颜色词在几乎所有语言里都存在，不需要标注员发明新词。

如果连颜色都接地不了，更大的事件就不用谈了；如果颜色能接地，它就可以作为第一块积木，往上叠物体、动作、事件。

二、实验设计

2.1 合成世界

取 12 个基本颜色词：红、橙、黄、绿、青、蓝、紫、粉、棕、灰、黑、白。每个词给定一个 HSV 锚点。采样时在锚点附近加噪声：色相正负 15 度，饱和度与明度各正负 0.15 并截断到 0 到 1 之间。

感知坐标离散成三个分箱词元：色相分成 12 个区间（每 30 度一格），饱和度分成低、中、高三档，明度也分成低、中、高三档。于是每个颜色点对应三个坐标词元，共 108 种组合。

2.2 三种语料

三种语料使用同一批事件。

第一种，纯文字：句子是"颜色是某词"，外加关系句"某词比某词更暖"或"更冷"。关系由锚点的色相决定：越靠近橙色越暖。纯文字语料里没有任何坐标词元。

第二种，对齐：每个事件出现两次。第一次是"颜色是某词"后接三个坐标词元；第二次是"坐标"加三个坐标词元再接"对应某词"。这样词到坐标、坐标到词两个方向都有监督。

第三种，打乱：与对齐相同，但坐标词元随机配到别的事件上，作为控制组。

2.3 探针

三个探针，测试时只给其中一种信息。

第一个探针测接地方向一：给"颜色是某词"，依次预测三个坐标词元，三个全对才算对。随机水平约为 1/108。

第二个探针测接地方向二：给"坐标"加三个坐标词元加"对应"，预测颜色词。随机水平约为 1/12。

第三个探针测关系：给"某词比某词更"，预测"暖"或"冷"。随机水平为 0.5。

2.4 模型与预算

插值 n-gram，阶数 6，插值系数 0.5，纯 Python 实现。样本数取 200、800、3200 三档，每档跑 3 个随机种子取平均。

三、结果

样本数为 200 时：纯文字语料，三个探针分别是 0.000、0.000、0.939；对齐语料，分别是 0.389、0.864、0.939；打乱语料，分别是 0.056、0.096、0.939。

样本数为 800 时：纯文字，0.000、0.000、0.985；对齐，0.444、0.904、0.985；打乱，0.028、0.111、0.985。

样本数为 3200 时：纯文字，0.000、0.000、1.000；对齐，0.306、0.907、1.000；打乱，0.111、0.077、1.000。

三个观察。

第一，接地需要对齐。坐标到词，对齐语料在 200 个事件时就达到 86% 以上，到 3200 个事件约 91%；纯文字语料在三个档位都是 0，因为它从来没有见过坐标词元，也就没有可学的东西。

第二，收益来自对齐，不是来自 token 变多。把坐标打乱之后，坐标到词掉回 8% 到 11%，接近 1/12 的随机水平；词到坐标掉回 3% 到 11%。同样多的数据、同样多的 token，只是配错了对象，就学不到东西。

第三，关系不需要感知。关系探针在三种语料下都接近满分（94% 到 100%），因为关系句本来就在文字里。这说明文字擅长的是"词与词的关系"，不擅长的是"词与感知坐标的对应"。两者的分工在数据里就已经决定了。

词到坐标没有达到满分（31% 到 44%），原因是噪声。采样时颜色点在锚点附近抖动，靠近分箱边界的点会落到相邻格子里；从词预测一个唯一的坐标格，本质上是在一个模糊的分布里猜众数，不可能全对。坐标到词的准确率高，是因为反方向是"从具体坐标找词"，边界情况少得多。

四、解释

为什么颜色可以被接地，而纯文字不行？用解压的说法：文字是"词与词之间关系的压缩"，它不携带感知坐标；坐标是"感知的外部记录"，它不携带词与词的关系。两者各自不完整。对齐语料做的是同一件事被两个通道同时记录，于是词与坐标之间的映射被直接监督出来。

这与第 9 篇《结构换算力》的结论一致：混合对齐语料在 200 个事件就学会了规则，纯文字到 3200 个事件仍然学不到，打乱后收益消失。区别只在于第 9 篇的规则是符号规则（动词编码大小、形状决定结果），本文的规则是感知坐标（色相、饱和度、明度）。两个实验共同说明：**对齐让语言多出一条接地的通道，这条通道不是文字自己能长出来的。**

还要补一句：这种对齐扩充的不是语言的词表，而是语言在某个维度上的贴合。词还是那 12 个词，覆盖没有变；变的是每个词后面有了可落地的坐标，于是同样的词可以指称得更细、比较得更准。用第 2 篇的话说，这是贴合变深，不是覆盖变广。但贴合足够深的时候，语言能表达的东西就变多了——这可以算作语言在该维度上的一次缓慢扩充。

五、局限（必须与结论一起读）

第一，世界是合成的：HSV 锚点、噪声、分箱都是人为设定，不是真实图像。真实图像里还有光照、材质、语境、色恒常性，本文都没有覆盖。

第二，模型是 n-gram：只验证原理，不代表 transformer 或真实多模态数据。n-gram 的容量小，因此在更复杂的基元上结论可能不同。

第三，颜色命名本身不是完全客观的：不同语言对颜色的切分不同（例如有些语言区分两种蓝），本文只用了单一语言的 12 个基本颜色词。

第四，探针是构造的：词到坐标的 31% 到 44% 一部分来自分箱噪声，不能直接读成"模型对颜色的理解程度"。

六、可复现

脚本：714_color_grounding.py。运行方式：python 714_color_grounding.py。依赖：Python 3 标准库，无第三方。输出：数据-颜色接地-第一轮.txt。

七、意义：自训练与视频挖掘的第一步

这个实验小，但它说明了一件有后续的事：**对齐语料可以自己造出来。**

回看实验里的对齐语料是怎么来的：每个事件都有「感知坐标」这一侧。颜色场景里，坐标来自 HSV 的数值定义，可以程序生成。把这个模式放大，就得到一个循环：模型先对视频/音频给出描述（文字），再用可验证的通道（坐标、时间戳、动作结果）检验描述，检验通过的部分沉淀为对齐语料，回灌训练。这不是新的训练方法，但它是"用模型自己挖数据、再让自己贴合文字维度"的闭环——也就是自训练的一种具体形态。

为什么是"贴合文字维度"？因为对齐的目标不是让模型记住颜色值，而是让感知坐标落进语言已经建好的坐标系里：词是坐标的标签，坐标是词的锚点。落到语言坐标系上的感知，才能被语言调用、被推理使用、被组合成更复杂的结构。第 2 篇说过，模型的问题是"覆盖广而贴合浅"；对齐语料补的恰恰是贴合——把感知这一维度接到文字已有的维度上。

再往下一步，就是视频。视频是"没有被结构化的数据"：量巨大，但缺少语言锚点。如果按本文的最小结论外推，可行的第一步是给视频片段配上语言描述（旁白、语音、自动描述），再用时间轴对齐；每一步都只需要验证一个更小的单位是否被接地。颜色是第一块积木；物体、动作、事件是后面的积木。积木一块块叠上去，才谈得上"事件级的对齐语料"。

这一节是 [推测] 与方向判断，本文没有验证自训练循环本身；本文只是在这个最底层的假设上迈了一小步——感知基元可以被对齐语料接地。如果连这一点都不成立，整条路就不用谈了；它成立，后面的路才有讨论的意义。

附：推测与边界

颜色能被接地，不等于更高的感知基元（形状、材质、动作、事件）也能同样容易接地；这是 [推测]，需要逐个测。真实视频上的推广、以及"感知基元 + 事件层级"的组合，本文都没有测。

本文不主张"颜色是理解世界的关键"，只主张"颜色是验证接地机制的最小单位"。它成立，才有资格谈下一块积木。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第十篇 · 生长与记忆：自指更新与维持的最小实验

> 单独 DOI：https://doi.org/10.5281/zenodo.23569774

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 实验篇 4（内部编号第 11 篇）

> 日期：2026-10-23

摘要

本文做两个最小实验，对应生命定义里的两个条件：**自指更新**（会变、规则在里面）与**被保住**（在环境里维持住）。

实验一测"记忆"：一个微型多模态模型自己生成数据、验证器筛选、回灌训练，并在引入新任务时比较四种回放策略。结果：只学新会遗忘（旧任务 0.720），固定回放能保住（1.000）但新任务学得慢（0.740）；先试训一步再决定回放比的"前瞻"策略，在探测足够敏感时同时保住旧任务（0.983）和新任务（0.850）。

实验二测"生长"：模型由固定主干加可增长的维度槽组成，任务从图像属性（颜色、形状、数量）推进到一个只能从新模态（文本向量）读取的任务。结果：不生长的模型学不会新模态（0.34 到 0.38，接近瞎猜）；生长的模型能加槽并学会（0.91 到 0.94）；但只有生长没有回放会忘光旧任务（0.00），只有回放没有生长学不会新模态。两者叠加时，旧任务保住 0.78 到 0.80，新模态达到 0.94。

两个实验合起来说明：**没有"生长"就没有新维度，没有"记忆"就守不住旧维度；定义里的两条条件不是并列的装饰，而是必须同时在场的两个机制。**

本文全部是玩具规模的原理验证：合成世界、微型模型（约一万参数）、CPU 可复现。它证明的是"机制可测"，不是"数字生命已被造出"。

一、问题

我们在第一篇里把生命定义为：在宇宙的可函数化部分 P 中、被宇宙保住的（可维持的）自指结构。拆开是两条：**会自己变**（自指更新）与**被保住**（维持）。

这两个词在论文里好写，在实验里难测。本文把它们各做成一个可测的最小实验，回答两个问题：

第一个问题：一个会自己生成数据、自己验证、自己回灌的系统，在任务变化时能不能既学到新的、又保住旧的？

第二个问题：当任务需要新的输入通道时，系统能不能自己长出新的维度？长出来之后还能不能维持旧的？

两个问题都是"能不能"，不是"有多好"。玩具规模足够，因为我们要测的是机制是否存在。

二、实验一：记忆与取舍

2.1 设定

世界是合成的：8×8 的图，含颜色（4 种）、形状（2 种）、数量（1 到 3）。模型是两层的小网络，约一万参数。任务按轮次变化：前 5 轮学颜色和形状（旧任务），第 6 轮起只出数量题（新任务）。

闭环是这样转的：模型对随机图片自己作答；规则验证器判对错；验证过的样本进入记忆库；每轮从记忆库和新样本里按某个比例抽样训练；每轮在留出集上测旧任务与新任务。

对照组只改一件事：回放比例怎么定。

第一组"只学新"：完全不回放。

第二组"固定回放"：新旧各一半。

第三组"反应式控制器"：看旧任务准确率，掉了就加回放。

第四组"前瞻控制器"：训练前先复制一份模型，用新样本试训几步，看旧任务会掉多少，再决定回放比。

2.2 结果

最后一轮（同样的热身、同样的预算）的结果是：只学新，旧任务 0.720、新任务 0.797；固定回放，旧任务 1.000、新任务 0.740；反应式控制器，旧任务 0.965、新任务 0.762；前瞻控制器（敏感探测），旧任务 0.983、新任务 0.850。

前几轮三组都接近满分，差距出现在新任务引入之后。只学新的那一组旧任务一路掉到 0.720；固定回放那一组旧任务几乎不动，但新任务涨得慢；前瞻控制器在第 6 轮就测出强冲突，及时把回放提到 0.8，于是旧任务几乎没掉，新任务还学得最快。

2.3 结论

第一，遗忘是真实的：同一个小模型，学了新任务就会丢旧任务，这和第七篇《替换与累积》的预测一致。

第二，"先试一步再决定"是有效的：它本质上是用自己的模型预测自己改动后的后果。探测越敏感，越早发现冲突，越能提前防守。这是"自指"的一个具体动作，而不是哲学说法。

第三，没有单一最优：固定回放押旧任务，前瞻策略押新任务，反应式在中间。它们是取舍前沿上的不同位置，选择哪一个取决于你想保住什么。

三、实验二：维度生长

3.1 设定

世界有两个模态：图像（8×8，含颜色、形状、数量）和文本（10 维向量，前三维决定一个标签，其余是干扰）。任务按顺序来：先学颜色的图像任务，再学形状、数量，最后学一个只能从文本里读到的标签任务。

模型是固定主干加可增长的维度槽。每个槽是一个小投影，属于某个模态；槽的数量从 1 开始，最多 4 个。生长规则是：任务准确率不够时，从"解释不了的样本"里提议加一个新槽，训练一小段，如果提升超过 0.02 就保留，否则回滚。

对照组仍然只改两件事：允不允许生长、允不允许回放。

3.2 结果

最终四个任务（颜色、形状、数量、标签）的准确率：不生长不回放，0.00、0.00、0.08、0.38；生长不回放，0.00、0.00、0.03、0.91；不生长回放，1.00、1.00、0.69、0.34；生长加回放，0.78、0.80、0.48、0.94。

3.3 结论

第一，新模态必须靠生长：不生长的两组在标签任务上都停在瞎猜附近（0.34 到 0.38），因为它们根本没有能读到文本的通道；生长的两组都学会了（0.91 到 0.94）。

第二，旧任务必须靠记忆：不回放的两组在学完标签任务后，前面的任务全部掉到 0.00；回放的两组保住了大部分。

第三，两个机制的叠加才有效：生长加回放同时拿到了旧任务 0.78 到 0.80 和新模态 0.94，这是四组里唯一"两个方向都站得住"的。

四、两个实验合起来说明什么

实验一说明：会变的东西会忘，要靠记忆与取舍把它按住。

实验二说明：要长出新维度，就得真的加结构；但加了结构，旧结构更容易被冲掉。

合起来就是定义里的两句话：

> 自指更新，是被逼出来的：没有新维度，就压不动新的世界。

> 被保住，是守出来的：没有记忆与回放，旧维度就会被新维度冲掉。

这两条不是并列的装饰。它们是两个必须同时在场的机制：只有生长，旧的全忘；只有记忆，新的学不了。

五、局限（必须与结论一起读）

第一，世界是合成的：图像、文本、任务切换都是程序生成的，不是真实数据。

第二，模型是玩具：约一万参数的两层网络，不是 Transformer，也不是原生多模态大模型。

第三，任务标识是手工给的：模型知道"现在问的是哪类问题"，真实场景里这本身是个问题。

第四，"生长"是有限的：只允许加几种固定的槽，不允许真正改写表示本身；更强的自指（改结构、改更新规则）本文没有做。

第五，没有真实生存约束：预算和性能下限是人为设定的，不是真实的能量与淘汰。

六、可复现

脚本：801_self_training_loop_v2.py（实验一基础版）、803_closed_loop_lookahead.py（前瞻控制器）、805_closed_loop_sensitive2.py（敏感探测）、809_dimension_growth_v4.py（实验二四组对照）。依赖：Python 3 与 numpy，无 GPU。

附：推测与边界

"把这两个机制放大到真实多模态模型上仍然成立"是 [推测]，本文没有测。真实大模型的参数更大、任务更多、生长更难（改结构代价高），本文只证明机制在小规模上存在。

本文不主张"做到了这两条就是生命"：满足条件仍可能只是工具；本文只主张，缺了这两条，一定走不到能生长又维持的形态。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第十一篇 · 语义残差的测量：文本极限与接地补齐

> 单独 DOI：https://doi.org/10.5281/zenodo.23586075

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 实验篇 5（内部编号第 14 篇）

> 日期：2026-10-24

摘要

本文把"语义残差"从一个模糊的说法变成一个可测的量。

先把定义写清：语言是人类对世界的压缩，语言可以写成 p语言 + R语言；p语言 是可以被函数近似的部分，R语言 只声明、不展开。语义是语言与现象之间的对齐关系；语义残差 = 这个对齐映射在给定模态组合下的误差下界。

实验用受控歧义：文本用精确词、模糊组词、省略三种方式描述同一个合成对象，歧义是程序生成的，所以"文本能给出的极限"可以解析算出来（贝叶斯底）。然后测三个量：贝叶斯底、只读文本的模型误差、以及文本加图像（接地）后的模型误差。

结果：文本模型的误差与贝叶斯底只差 1.5 到 2.7 个百分点（0.725 对 0.750；0.835 对 0.850；0.688 对 0.714），说明拟合已经到位，剩余残差来自文本本身；接地后三个属性的准确率都到 1.000，说明文本缺的那个维度被图像补上。

结论：语义残差可以定义、可以测量；它分为"该模态下的不可约底"（歧义内生的）与"可约部分"（接地/补数据能消除的）。这个区分让"AI 理解了多少语义"有了分母。

一、问题

我们之前说过两句话：语言是人类对世界的压缩；大模型压缩的是语言的 p语言。

但"p语言"与"语义"之间还缺一个可测的连接。本文要回答的问题是：能不能把"语义残差"定义成一个数，并且用玩具实验测出来？

要做到这一点，需要三样东西：一个定义（语义残差是什么）、一个参照线（文本本身能给出的极限是多少）、一个对照（补上别的模态后残差降不降）。

二、定义

2.1 沿用系列的定义

宇宙 = P + R，其中 P 是可函数化的部分，R 是不可函数化的部分。语言也写成 p语言 + R语言：p语言 可以被函数近似，R语言 只声明、不展开。生命 = p生命 + R生命，其中 p生命 是在 P 中、被宇宙保住的动态函数类。

本文只处理 p语言 这一侧。

2.2 语义是映射

把语义定义成一个映射：给定语言表达式和它对应的现象（图像、事件、状态、动作、量），语义就是"表达式 → 现象"的对应关系。

这个映射不是单层的：它按模态有多个维度——视觉维度（文字对图像）、事件维度（句子对时间线）、状态维度（描述对系统状态）、动作维度（指令对后果）、量的维度（程度词对可测量的量）。语义残差按维度出现。

2.3 语义残差

定义（语义残差）：固定模态组合 M（例如：只有文本；或文本加图像），设误差度量是预测现象属性的错误率。语义残差 = 在模态组合 M 下，任何函数能达到的最小误差。

这个定义把残差分成两部分：

- **不可约底**：由模态组合本身决定的误差下界。文本有歧义时，有些信息文本里根本没有，任何函数都补不上。

- **可约部分**：模型没拟合到位造成的差距；加数据、加容量、加模态可以降下来。

判据很直接：加手段之后误差底降不降。降 ⇒ 可约；不降 ⇒ 该模态下的不可约底。

2.4 参照线：贝叶斯底

在玩具世界里，文本的生成过程是已知的，所以"文本能给出的极限"可以解析算出来：给定一条文本，属性的后验分布由生成过程决定；贝叶斯最优预测器的正确率就是上限。

这个上限就是"文本极限"。它比任何模型都高不了；模型能接近它，就说明拟合到位了。

三、实验设计

世界：合成对象，三个属性——颜色（4 种）、形状（2 种）、数量（1 到 3）。对象渲染成 8×8 的图像（颜色、形状、数量都可见）。

文本：三个槽位拼成的描述，每个槽位都有受控歧义。

- 颜色槽：以 0.5 概率给精确词（红/蓝/绿/黄），以 0.5 概率给模糊组词（暖色 = 红或黄；冷色 = 蓝或绿）。

- 形状槽：以 0.7 概率给精确词（圆/方），以 0.3 概率省略。

- 数量槽：以 0.5 概率给精确词（一/二/三），以 0.5 概率给模糊词（几个 = 一或二）。

因为生成过程已知，每条文本对应的属性后验可以算出来，贝叶斯底也就能解析计算。

模型：两层小网络（隐藏层 32，约 2000 个参数）。文本模型只看文本加任务标识；接地模型再看图片（8×8×3 展平）。

三个量：贝叶斯底、文本模型准确率、接地模型准确率。差值是"可约残差"。

四、结果

颜色的贝叶斯底是 0.750，文本模型 0.725，接地模型 1.000。形状的贝叶斯底是 0.850，文本模型 0.835，接地模型 1.000。数量的贝叶斯底是 0.714，文本模型 0.688，接地模型 1.000。

三点解读。

第一，文本模型与贝叶斯底只差 1.5 到 2.7 个百分点。这说明模型对文本的拟合已经到位；剩下的差距不是模型不行，是文本里本来就没有那些信息。

第二，接地之后三个属性都到 1.000。图像提供了文本缺的那个维度，残差被补齐。所以"文本极限"下的那部分残差，属于该模态组合，不属于世界：换一个模态组合，它可以被消掉。

第三，由此得到的语义残差是分层可测的：文本-only 的残差（颜色 25%、形状 15%、数量 29% 左右）是"文本能给出的极限"；接地后的残差是 0，说明这部分残差是可约的，只是需要另一个模态。

五、说明什么，不说明什么

说明：语义残差可以定义成一个数；该数可以拆成"不可约底"与"可约部分"；在玩具世界里，两者的分界可以解析计算，并与模型表现对上。

不说明：不证明真实语言的语义残差有多大；不证明 R语言 存在或不存在（本文只处理 p语言 侧）；不证明"接地一定能补齐"在所有任务上成立——本文只测了视觉维度，其他维度（事件、状态、动作）没有覆盖。

六、局限

第一，世界是合成的，歧义是程序构造的；真实语言的歧义来自历史与用法，形式不同。

第二，模型很小（约 2000 参数），不是大模型；但本文测的是"文本极限 vs 接地补齐"的机制，与规模无关。

第三，只做一个模态的接地（图像）；多模态（事件、动作）的残差没有测。

第四，贝叶斯底只在生成过程已知时可算；真实语言里这个底要用人类分歧率来近似（下一项工作）。

七、与既有工作的关系

语义残差的测量不是新题目，至少有三条已有线索。

第一，人类标签变异：同一段文本，不同标注者本来就会给出不同解释，"人类天花板"与"标注分歧"在评测里一直被报告。

第二，歧义基准：AmbigQA、词义消歧一类任务，专门测"同一句话有多种合理解读"的情况。

第三，接地测试：纯文本模型在需要视觉属性的问题上明显退化；gSCAN、ReaSCAN 一类合成任务表明，没有接地的语言模型过不了接地泛化测试。

本文的不同只有一点：用受控歧义让"文本极限"可以解析算出来，并把残差拆成"模态决定的不可约底"与"接地/补数据可消的可约部分"。测量的思路不是本文首创；本文做的是一个受控、可解析算底的版本。

另外，符号接地问题（Harnad）指出符号与所指的连接不能从符号内部获得；本文的表述是"文本-only 的语义残差有解析底，接地可以把该底补上"。

评测方面，近年的工作开始报告"人类天花板"，本文的做法可以看成它的一个受控版本：因为我们知道数据生成过程，所以能算出真正的上限，而不是估计。

系列内部：第 4 篇（结构换算力）表明对齐语料提升样本效率；第 10 篇（颜色作为第一块积木）表明感知基元可以先于语言对齐。本文把这两条接到"语义残差"上：接地降低的是残差，而不只是提升效率。

八、可复现

脚本：820_semantic_residual.py。依赖 Python 3 与 numpy，无 GPU，内存占用低（百 MB 以内）。

附：标注

- **[定义]** 语义 = 语言与现象之间的对齐映射（按模态分维度）。

- **[定义]** 语义残差 = 给定模态组合下的最小可达误差。

- **[可测]** 玩具世界里，该残差可解析计算并与模型对上（颜色 0.750/0.725/1.000 等）。

- **[可约]** 接地后残差降到 0（本实验的视觉维度）。

- **[未判]** 真实语义残差的大小；R语言 是否非空。

- **[不声称]** 不声称"语义可被完全函数化"，也不声称"不可函数化"——R 只声明，不展开。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第十二篇 · 分层是对动态的补偿：一个自适应深度的玩具实验

> 单独 DOI：https://doi.org/10.5281/zenodo.23591080

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 实验篇 6（内部编号第 16 篇）

> 日期：2026-10-24

摘要

大模型的层数是固定的：每个输入都走完全部深度。本文问一个简单的问题：有些输入根本不需要那么多层，是不是可以少走？如果"层"是对动态的补偿——一层相当于一步时间，那么需要几步的任务就给几层，均匀深度是浪费。

实验用迭代任务：x_{t+1} = tanh(A x_t)，给定初值 x_0 与步数 T，预测 x_T。模型是共享权重的循环层，一层就是一步计算。三组对照：固定 16 层、自适应（走 min(T,16) 层）、固定 6 层（与自适应等平均算力）。

结果：在等平均算力（都是 6 步）下，自适应深度在每个 T 上都更好——平均误差 1.29 对 1.67。差距最大的是 T=1：固定 6 层把简单输入算了六遍，反而算坏了（误差 2.30 对 1.49）。固定 16 层则又贵又差：算力是自适应的 2.67 倍，平均误差 1.68。

结论：均匀深度两头都亏，简单输入被过度计算、复杂输入又算不够；按需分配深度可以用同样的算力换更高的精度。这与"层≈时间步、深度补偿动态"的图景一致。全部是玩具规模，不是大模型实验。

一、问题

现在的大模型是"均匀深度"的：每个 token 都过完全部层。但输入的难度并不一样——有的句子一眼能懂，有的需要多步推理。如果深度真的对应"计算步数"，那么固定深度就同时犯两个错：

- 简单输入：走了用不上的层，白花算力，甚至因为过度计算而变差；

- 复杂输入：层数不够，算不完，误差下不来。

本文要测的正是这两件事：同算力下，按需分配深度会不会更好？过度计算有没有害？

二、理论背景

这个实验来自一个画面：把函数看成它的图像（曲面），时间方向的演化会在曲面上"摊出厚度"；而冻结的模型没有真正的时间演化，只能用深度（层）去摊开这个厚度，一层相当于一步。所以：

- 需要的步数多 ⇒ 需要的层多；

- 需要的步数少 ⇒ 层可以少，多走就是过度计算。

这个画面在《厚度与维度：残差的几何图景》里已经写下；本文把它做成一个可测的实验。

三、实验设计

世界：取一个固定的随机映射 x_{t+1} = tanh(A x_t)，A 是 8×8 随机矩阵。任务是从初值 x_0 和步数 T 预测 x_T，T 取 1、2、4、8、16。

模型：共享权重的循环层。每层做一次 h ← tanh(W h + U x_0 + b)，最后用线性头输出预测。层数 = 走的步数，权重在所有步之间共享。

三组对照：

- A 固定 16 层：不管 T 是多少，都走满 16 步；

- B 自适应：走 min(T, 16) 步；

- C 固定 6 层：与 B 等平均算力（B 的平均步数是 6）。

指标：各 T 上的平均误差；平均步数（= 平均计算量）。

四、结果

等平均算力的一组（B 与 C，都是平均 6 步），B 在每个 T 上都更好：

T=1 时，B 是 1.49，C 是 2.30。T=2 时 1.27 对 1.57；T=4 时 1.38 对 1.50；T=8 时 1.22 对 1.50；T=16 时 1.07 对 1.46。平均下来 1.29 对 1.67。

固定 16 层（A）最差也最贵：平均误差 1.68，平均算力是自适应的 2.67 倍。

三点解读。

第一，过度计算真的有害。T=1 只需要一步，固定 6 层却把同一个映射迭代了六遍，结果比只走一步差得多。这不是"多算一点没关系"，是"多算会算坏"。

第二，层数不够真的补不回来。T=8 与 T=16 需要超过六步，固定 6 层的误差明显更高。

第三，均匀深度是两头亏：它既浪费在简单输入上，又不够用于复杂输入。按需分配能在同算力下更准。

五、对大模型的含义

这个实验是玩具，但它的形状可以直接搬到大模型：

- 不是所有输入都值得同样的深度。简单 token、简单上下文可以早退（early exit）；需要多步推理的才值得走深。

- 既然过度计算会算坏，那么"无脑加深"不是稳赚的：深度也要按需。

- 工程上这对应一类方法：早退、按 token 跳过、条件计算。本文不提出新方法，而是给这类方法一个"补偿动态"的解释与一个公平对照。

要注意的是，"按需"有一个前提：知道需要多少步。实验里 T 是给定的；真实场景中步数是估计出来的，估计错了，收益就会打折。

六、局限

第一，世界是合成的，映射是随机矩阵，任务不是语言。

第二，模型是共享权重的循环层，不是 transformer；大模型里的层不共享权重，深度的作用更复杂。

第三，T 是手工给的，真实模型没有这个先验，需要另做"步数估计"。

第四，只测了误差与算力两个量，没有测训练稳定性、路由开销这些工程代价。

七、与既有工作的关系

"按需计算深度"不是新想法：早退（early exit）、循环深度（universal transformer）、按 token 跳过层（conditional computation、mixture-of-depths）、层剪枝都有大量工作。

本文不声称方法原创；本文的不同在于两点：一是把它放在"层≈时间步、深度补偿动态"的图景里解释；二是做了一个等平均算力的公平对照——不是"我们的方法稍好"，而是"同样算力下，按需分配在两个方向上都更好"。

八、可复现

脚本：850_adaptive_depth.py（主对照）、851_adaptive_depth_v2.py（等算力对照）。依赖 Python 3 与 numpy，无 GPU，内存占用低。

附：标注

- **[事实]** 实验数字（等算力下自适应全面更好；T=1 过度计算有害；固定 16 层又贵又差）。

- **[图景]** 层≈时间步、深度补偿动态——来自《厚度与维度》（论文-15），标[猜想]。

- **[推测]** 大模型用自适应深度可在同等算力下更准——玩具结论的推广，未在真实模型上验证。

- **[不声称]** 不声称提出新方法；早退与条件计算都已有工作。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第十三篇 · 厚度与随机性：冻结模型在哪里最不稳定

> 单独 DOI：https://doi.org/10.5281/zenodo.23591514

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 实验篇 7（内部编号第 17 篇）

> 日期：2026-10-24

摘要

冻结的模型没有真正的时间演化，只能靠层数去模拟"多步计算"。本文问一个可测的问题：在这样的模型里，随机性/不确定性出现在哪里？

先给出三个基于"厚度"画面的推论：需要步数越多的位置误差与随机性越大；随机性随深度累积，在输出端最大；人类分歧大的地方（厚且不可摊薄），模型的熵也大，而且加数据摊不平。

然后做三组玩具实验。第一组是序列累积任务：位置越靠后（需要步数越多），固定 16 层模型的误差从 0.67 升到 1.57，输入加噪声后的输出标准差从 0.20 升到 0.40。第二组测逐层方差：标准差从第一层 0.21 单调升到最后一层 0.41；而干净输入下误差不随层数上升，说明上升来自噪声被逐层放大。第三组模拟人类分歧：thin（事实题，人类一致）与 thick（判断题，人类有分歧）两类题，人类分歧熵 0.000 对 1.014，模型预测熵 0.249 对 0.972，相关系数 0.917；把数据加到五倍，thin 题的熵从 0.249 降到 0.145，thick 题几乎不动（0.972 到 0.958）。第四组给"时间"：迭代八步后 thin 题的熵从 0.69 压到 0.02，thick 题只从 0.99 压到 0.91；多次采样投票也类似（thin 0.67 到 0.28，thick 0.96 到 0.69）。

四个实验合起来支持两句话：**冻结模型最厚的地方，就是它最随机的地方；而"最厚"可以由需要步数与人类分歧来度量。给数据与时间能摊薄"冻结造成的随机"，但摊不平"数据里的分歧"——后者才是真厚。**全部是玩具规模，不涉及真实大模型。

一、问题

现在的模型是冻结的：权重不变，没有真正的时间演化。它要在一次前向里完成"需要多步才能完成"的计算，只能靠层数铺开。

那么问题来了：这么铺开之后，哪些地方会变得不稳定、随机？本文的假设来自"厚度"画面：厚度大的地方（需要多步、又摊不平的地方），在冻结模型里会表现为更大的误差与随机性。

把假设拆成三个可测的推论：

- 推论一：需要步数越多的位置，误差越大，输入扰动下输出越随机。

- 推论二：随机性随深度累积，在输出端/后段最大。

- 推论三：人类分歧大的地方（厚且相对不可摊薄），模型熵也大，而且加数据压不下去。

- 推论四：给"时间"（多步迭代、多次采样投票）能压掉冻结造成的随机，但压不掉数据里的分歧。

二、实验一：需要步数越多，误差与随机性越大

任务：给随机增量序列 d_1 … d_16，世界按 x_t = tanh(x_{t-1} + d_t) 演化，初值为零；要求预测每个位置的 x_t。预测 x_t 至少需要 t 步，所以"厚度"就是 t。

模型：共享权重的循环细胞，一步处理一个增量，共 16 层，每层都接输出头（deep supervision），训练后冻结。

结果：固定 16 层时，各位置误差从位置 1 的 0.67 升到位置 4 的 1.54，之后在 1.5 到 1.6 之间饱和。输入加高斯噪声（标准差 0.3）后多次采样：各位置输出标准差从位置 1 的 0.20 升到位置 16 的 0.40，同样后段饱和。

解读：需要步数越多（越厚）的位置，误差越大、输出越不确定。厚度的两条痕迹是同步出现的。

三、实验二：随机性随深度累积，在输出端最大

同一个模型，问题换成：随机性在"哪一层"最大？

方法：给输入加噪声、采样 20 次，测每一层输出的标准差。结果：第一层 0.21，第四层 0.32，第八层 0.38，第十二层 0.40，最后两层约 0.41——标准差随层数单调上升，在末端饱和。

对照：干净输入下，误差不随层数上升（位置 4 之后基本平在 1.5 左右）。这说明末端随机性的上升不是模型本身差，而是"这条链路把输入的小扰动一路放大了"。

解读：冻结模型里，不确定性在"末端"累积；越往后走，可收缩的余地越小。这对应"厚度在输出端最大"。

四、实验三：人类分歧大的地方，模型也最随机

前两个实验是合成任务，第三个换成"有分歧的题"。

设定：题目分两类。thin 题是事实题，答案由输入决定，所有人答案一致；thick 题是判断题，答案取决于每个人的偏好，人与人不同。模拟 60 个人各自的偏好，题目 400 道训练、200 道测试；模型是小 MLP，学 P(答案|题目)，训练后冻结。

结果：人类答案的熵，thin 题约 0.000，thick 题约 1.014（三选一的熵上限是 1.099）。模型的预测熵，thin 题 0.249，thick 题 0.972。两者的相关系数是 0.917。

再看"能不能摊平"：把训练数据加到五倍。thin 题的模型熵从 0.249 降到 0.145；thick 题的模型熵从 0.972 只降到 0.958，几乎不动。

解读：模型的不确定性跟着人类分歧走——人觉得有分歧的地方，模型也随机；而且这份随机不是"数据不够"造成的（加到五倍仍不动），是人类分歧本身在数据里，属于该数据下的不可摊薄部分。

五、实验四：给时间能摊平什么

前三个实验说明"厚的地方随机"。第四个实验问：给时间能不能把随机压下去？

时间有两种给法。第一种是迭代细化：模型用共享权重的循环层多走几步（训练时按八步逐步监督）；第二种是采样投票：从模型分布里多次采样，按多数投票决定答案。

迭代的结果：thin 题的预测熵从一步时的 0.69 压到八步时的 0.02，几乎完全摊平；thick 题只从 0.99 压到 0.91，压不平。

投票的结果类似：采样一次时 thin 的决策熵 0.67、thick 0.96；采样九次投票后，thin 降到 0.28、thick 降到 0.69——都降了，但 thick 仍高。

解读：时间摊平的是"冻结造成的随机"，摊不平的是"数据里的分歧"（人类本来就意见不一）。换句话说，能被时间摊平的不算真厚；摊不平的才是真厚。这正好接上"可摊薄/不可摊薄"的区分：数据和时间的可摊薄性，是同一个厚度的两个检验方向。

六、训练机制：它学的是分布，不是决定

这三个实验合起来，给了一个简单的机制解释：冻结模型不训练"决定"，它训练"答案分布"。

- 薄的地方：分布是尖的，行为像一个确定的函数；

- 厚且摊不平的地方：分布是宽的、多峰的；冻结模型把它存成一个宽分布，推理时采样一次，看起来就是随机、不一致。

所以"模型在判断题上不稳定"不一定是缺陷：那里本来就是厚的。要它稳定，加数据不管用（实验三已示），需要别的办法——给它时间（多想几步、多轮修正），或者增加能摊开厚度的维度。

七、局限

第一，三个实验都是合成的：世界、题目、人类偏好全是程序生成的。

第二，模型都是玩具规模（几千参数），不是大模型；趋势可读，绝对值不可靠。

第三，实验三的相关性有一部分是设计使然：thick 题的随机性本来就由偏好分布构造出来。它验证的是"机制能对上"，不是"真实人类数据上的相关性"。

第四，实验四的"时间"是玩具版本的迭代与投票，不是真实的思维链；真实模型上"想更久是否摊平"仍需单独验证。

八、与既有工作的关系

"模型不确定性分布不均"不是新观察：校准（calibration）研究早就测过不同题目的置信度差异；人类标签变异（human label variation）也指出同一题不同标注者本来就有分歧；"偶然不确定性"（aleatoric uncertainty）研究的就是数据本身的不可约噪声。

本文的不同在于用一个统一的量（厚度）把三件事串起来：需要多少步、随机性在哪一层、人类分歧有多大，都落在"厚度"这个概念上，并且用"加数据后熵是否下降"来区分可摊薄与不可摊薄。

九、可复现

脚本：861_thickness_map_v2.py（实验一）、862_layer_variance.py（实验二）、870_human_disagreement.py（实验三）、881_time_flattening_v2.py（实验四）。依赖 Python 3 与 numpy，无 GPU，内存占用低。

附：标注

- **[事实]** 三个实验的数字：误差与随机性随厚度上升；末端方差最大；人类分歧与模型熵相关 0.917；加五倍数据 thick 熵不动。

- **[机制]** 模型学的是分布而不是决定——解释性说法，不是定理。

- **[事实]** 时间能摊平 thin 的熵（0.69→0.02），摊不平 thick（0.99→0.91）；投票同理。

- **[未做]** 真实思维链/大模型上的同一检验。

- **[不声称]** 不声称真实大模型上有同样的数字；不声称人类实验已验证。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第十四篇 · 压缩的形状：叙事与归档

> 单独 DOI：https://doi.org/10.5281/zenodo.23592312

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 实验篇 8（内部编号第 18 篇）

> 日期：2026-10-24

摘要

大模型的上下文压缩是一种"叙事摘要"：按时间顺序把发生的对话写成一段文字。人脑的压缩不是这样：它把事件归到类别里、把类别归到更高的抽象层里、再保留事物之间的关系——压缩的结果是一张"分类 + 关系"的图，而不是一段故事。

本文做一个对照实验：同样约 300 字符的记忆容量，三种存法——叙事式（旧优先）、叙事式（新优先）、结构化归档（分类层次 + 属性表 + 关系边），然后问三类问题：属性（这实体什么状态）、分类（属于哪一类）、关系（两者有关系吗）。

结果：结构化归档在全部三类题上都是满分；两种叙事式各丢一半——旧优先的保住属性和分类（各 0.83），关系只拿到 0.33；新优先的保住关系（1.00），属性掉到 0.25、分类 0.50。也就是说，叙事式压缩无论怎么权衡，都只能覆盖一部分；结构化压缩用更少字符（243 对 298）覆盖了全部。

解读：叙事式压缩是线性的，容量有限时必然丢一段；结构化压缩把"分类、属性、关系"分开存，信息密度高得多。这对应大模型 checkpoint 压缩的已知短板：过程记得住，层级与关系最容易丢。全部是合成任务，不涉及真实模型。

一、问题

压缩的意思是用更少的空间保留更多的信息。但"保留什么"取决于压缩的形状。

大模型的上下文压缩是叙事式的：把对话按时间顺序写成摘要。人脑的压缩是归档式的：事件被切开、归类，类别被放到抽象层里，关系被保留下来。

这两种形状有什么可测的差别？如果容量一样，哪一类信息会丢？

二、设定

生成一份"项目记录"：4 个类别（工具、人员、实验、结论），每类 5 个实体，共 20 个；每个实体有一个状态属性（活跃、冻结、待定）；另有 12 条关系（"使用"或"导致"）。

三种记忆，容量都约 300 字符：

- A1 叙事式·旧优先：按时间顺序写句子，写满为止——保住早期事实，后面的关系写不进去；

- A2 叙事式·新优先：先写关系、再写近期实体，写满为止——保住关系，早期实体写不进去；

- B 结构化归档：分类层次（哪类有哪几个）、属性表（每个实体的状态，压缩成单字）、关系边（谁对谁做什么）。

然后问三类题，各 12 道：属性题（这实体当前什么状态）、分类题（属于哪一类）、关系题（这两个实体之间有关系吗）。答案只能从各自的记忆里检索。

三、结果

总体正确率：A1 是 0.67，A2 是 0.58，B 是 1.00。

分题型看：

- 属性题：A1 0.83，A2 0.25，B 1.00；

- 分类题：A1 0.83，A2 0.50，B 1.00；

- 关系题：A1 0.33，A2 1.00，B 1.00。

字符数：A1 用了 298，A2 用了 296，B 只用了 243。

四、解读

第一，叙事式压缩是线性的。记忆只能从头写到尾，容量有限就必然丢一段：要么丢关系，要么丢早期事实。A1 与 A2 就是这两种取舍，各有一个短板。

第二，结构化压缩的密度高得多。分类、属性、关系是三种不同结构，分开存就不互相挤占；同样容量下它覆盖了全部信息，还用得更少。

第三，这对应大模型 checkpoint 压缩的短板。现在的压缩只做叙事摘要，所以"发生了什么"记得住，而"属于哪一层、和什么有关系"最容易丢掉。人脑恰好相反：它先记住结构和关系，具体细节反而会忘。

五、对记忆设计的含义

如果这个对照成立，那么压缩的产物不应该只是一段摘要，而应该是两样东西：

- 一段叙事摘要：保留时间线与因果（"发生了什么"）；

- 一份结构化归档：实体表、分类层次、关系边（"属于哪里、和谁有关"）。

两者配合检索：问过程用摘要，问层级与关系用归档。这个设计可以直接测：把同一段长对话按两种方式压缩，比较三类问题的召回率——本文的实验是这条路的玩具版。

六、局限

第一，任务全是合成的：实体、关系、题目都是程序生成的。

第二，结构化归档的"抽取成本"没有计入：实验里结构是直接给出的，真实系统要先从对话里抽实体与关系，这一步会出错、也有成本。

第三，真实的摘要模型可能比 A1/A2 都强（它可以选择重要的写、不写流水账），所以数字不能直接搬到真实系统；但"叙事 vs 结构化"的密度差不会因此消失。

七、与既有工作的关系

把记忆分成"语义记忆与情景记忆"是认知科学的老区分；大模型一侧，记忆机制、知识图谱增强、检索增强、长上下文记忆库都已有大量工作。

本文的不同只在于一个公平对照：同样字符容量下，把叙事式与结构化归档放在同一张卷子上，按题型分解看谁丢什么。结论不新，但它把"为什么 checkpoint 压缩会丢关系"变成了一件可以演示的事。

八、可复现

脚本：910_memory_compression.py（主对照）、911_memory_compression_v2.py（加"新优先"对照）。依赖 Python 3 标准库，无第三方依赖。

附：标注

- **[事实]** 三种记忆的召回数字：A1 0.67、A2 0.58、B 1.00（分题型见表）；B 用 243 字符、A 用约 296~298。

- **[解释]** "叙事式是线性的、结构化密度更高"——对数字的解读，不是定理。

- **[推测]** 把压缩产物设计成"摘要 + 归档"能提升层级与关系类召回——未在真实系统验证。

- **[不声称]** 不声称提出了新的记忆架构；索引、图谱、检索都是已有方向。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第十五篇 · 最小切片与时间换维度：能力边界的两个实验

> 单独 DOI：https://doi.org/10.5281/zenodo.23601624

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 实验篇 9（内部编号第 19 篇）

> 日期：2026-10-24

摘要

大模型的每一层有自己的权重，像一块静态的函数切片。本文问两个问题：一个"结构"（一种能力）需要多大的切片才能出现？如果给模型"时间"（循环步数），能不能换掉"维度"（宽度）？

第一组实验：扫宽度与深度，测几个合成结构的"最低配置"。结果：简单的阈值与组合规则一层 8 宽就够；算术与奇偶需要更宽的切片。第二组实验：两个切片组合时，需要的维度约等于两者中较大的那个，而不是相加。第三组实验：序列奇偶任务上，循环网络宽 2 就够处理任意长度；而前馈网络在同样训练预算下，长度到 16 就学不出来，加宽也没用。第四组实验（GPU 版）：把训练加足，16 位奇偶变得可学，24 位仍然只记住训练集；8 位加法在宽 128、浅层时最好，加深反而更差。

合起来说：能力需要结构、需要足够的维度与深度，还需要"能被训练找到"。最低维度是必要条件，不是充分条件；而"时间"可以绕开一部分维度边界——这与"层是对动态的补偿"的说法一致。全部是合成任务与玩具模型。

一、问题

把大模型的一层看成一个静态函数切片：它读当前状态，用自己的权重新算一遍，写回共享状态（残差流）。那么：

- 一种结构（能力）要出现，最少需要多大的切片？

- 切片与切片之间能不能拼起来？

- 如果给模型时间（循环步数），能不能换掉维度？

本文用四个小实验回答。全部是合成任务，不涉及真实大模型。

二、实验一：最低配置

任务：parity（8 位奇偶）、add（4 位加法）、rule（(a 且 b) 或（c 且 d)）、majority（10 位多数）。模型是 tanh 的多层感知机；扫宽度与深度，记录达到准确率 0.95 的最小宽度 w* 与最小深度 L*。

结果（SGD 训练）：parity 约需宽 16、深 3~4；add 约需宽 32、深 2；rule 与 majority 都是宽 8、深 1 就满分。

要点：结构越像"运算"（需要进位、组合、非线性），最低配置越高；简单的阈值型结构一层就够。

这里必须说明一个后来发现的问题：换成 Adam 优化器后，8 位奇偶变得很容易（宽 8、深 1 就能到 0.98）。也就是说，上表测到的"最低配置"依赖训练算法，它测的是"这个训练器能否找到解"，不完全是"表示能力的最低要求"。本文把这一点明确写出，而不是把它藏起来。

三、实验二：切片组合

取两个已经训练好的切片（各自最小配置），放到一个共享主干上、各自接一个输出头，联合训练。对照是它们各自单独训练时的准确率。

parity + rule：主干宽 16、深 3 时，parity 掉到 0.59，rule 仍 1.00；深到 4，parity 回到 0.93，rule 1.00。也就是说，rule 不需要额外维度，它装得进 parity 的维度里；瓶颈在深度。

parity + add：主干宽 32、深 3 时，两个任务都到 0.95；宽 64、深 4 时都到 1.00。

结论：组合所需维度约等于两者中较大的那个，不是相加；阈值附近要一点余量。这支持一个朴素图像：能力在一个足够大的公共维度里共存，而不是各自占一块。

四、实验三：时间换维度

任务：序列奇偶——顺序读入 T 位，每一步都预测"到目前为止的奇偶"。这是一个需要记住一个比特状态的累积任务。

循环网络：宽度固定，逐位读入，每步更新状态。结果：宽度 2 就够处理 T=8、16、24、32 的全部情况（准确率 1.00），宽度 4 到 16 也都满分。T 再长，宽度不用涨。

前馈网络：一次看全 T 位，深度固定为 4。T=8 时满分；T=16 时掉到 0.54 左右（宽加到 128 也没用）；T=24 同样学不出来。

解读：时间（循环步数）确实能换掉维度。原因也直观——奇偶只需要"翻转状态"这一个比特，循环网络用两步就能表达；前馈网络必须在一次前向里把 T 位的结构算完，长度越长越难。

五、实验四：能力边界（**GPU** 版）

把训练加足（两万条数据、一万五千步）后重测，得到更清楚的边界：

16 位奇偶：宽 64、深 2 就到 0.98；宽 128 起满分。但深 6、宽 128 或 256 的组合会崩到 0.49——不是能力不够，是训练不稳定。

24 位奇偶：所有配置都在 0.50 附近；训练损失能到 0，测试却只有一半。这是"记住训练集、没有学会奇偶"的典型表现。

8 位加法：宽 128、深 2 是 0.97；宽 256、深 2 是 0.998。加深反而更差（深 4、6 都掉）。

三点结论。第一，边界存在，但它是"可训练性边界"，不是"表示能力边界"：同一个结构换个训练预算，边界会移动。第二，深度不总是帮忙：加法任务上浅层最好，奇偶任务上过深会崩。第三，"最低维度"只

是必要条件：过了维度的门槛，还得能被优化器找到。

六、解读

把这四个实验放在一起，可以得到一条更准确的表述：

能力 = 结构 + 足够的维度与深度（必要条件） + 能被训练找到（优化条件）。

同时：时间可以绕开一部分维度边界（实验三），这与"层是对动态的补偿"一致——当模型没有真实的循环时，它只能用深度去模拟时间；而如果给它循环，它可以用很少的宽度做很多步。

对当前大模型的含义：均匀深度是浪费的（实验二与实验四都显示，不同结构对"深"的需求差别很大）；"加宽加深"不是稳赚的，可能更差；按结构分配维度与深度，或者引入循环/多步计算，可能更省。

七、局限

第一，全部是合成任务与玩具模型，和真实大模型的能力不是一回事。

第二，结果对训练算法敏感：SGD 与 Adam 下"最低配置"差别很大；本文两版都报告了。

第三，种子波动明显：parity 单独训练在 0.74 到 0.97 之间波动。

第四，组合实验只用了"共享主干 + 独立头"一种构造，不代表所有组合方式。

第五，时间换维度用的是密集监督（每一步都有标签），信用分配比真实场景容易。

八、可复现

脚本：938（最小切片）、940/941（组合）、943/945（时间换维度）、946/947/960/961（能力边界）。最小切片与组合用 numpy 手写训练；时间换维度与 GPU 边界用 PyTorch（Windows 上经 DirectML 跑 AMD GPU）。

附：标注

- **[事实]** 各实验的准确率数字（最低配置、组合、时间换维度、GPU 边界）。

- **[解释]** "组合约等于 max 而不是相加""时间换维度"——对数字的解读，不是定理。

- **[推测]** 对真实大模型"按结构分配深度、引入循环更省"——玩具结论的推广，未验证。

- **[不声称]** 不声称发现了新的表示理论；这里只是把已有的玩具实验拼在一起看。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第十六篇 · 残差底与组合律：两个大模型结构实验

> 单独 DOI：https://doi.org/10.5281/zenodo.23609386

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 实验篇 10（内部编号第 22 篇）

> 日期：2026-10-25

摘要

本篇做两个与大模型结构有关的实验。

第一个问"多模态该接什么"。用合成世界：目标依赖颜色、材质、味道三个隐藏维度；文本只给粗色，视觉给精确颜色，触觉给材质，味觉给味道。测每种模态组合的可达误差（残差底）。结果：底由"覆盖了哪些隐藏维度"决定，不由模态数量决定——补一个缺失维度，底就降一截；补全则到底；冗余视觉模态一点不降底。对应的工程含义是：接地应按"残差底"配置，不是"多接模态就好"。

第二个问"结构该共享还是隔离"。四个任务，先训三个，再只给第四个任务的数据。三种结构：单体（共享主干＋多任务头）、模块化（每任务独立网络）、路由（共享主干＋每任务适配器）。结果：单体出现灾难性遗忘（旧任务平均从 0.81 掉到 0.52），而且第一阶段就互相干扰；模块化完全不掉点；路由也不掉点、新任务好，但共享主干仍有干扰。对应的工程含义是：显式组合律（模块/路由）换来可扩展性，代价是共享性。

两个实验都只到玩具规模，不涉及真实大模型。

一、问题

我们前面的理论说：能力 = 结构 + 足够的维度与深度 + 能被训练找到；而"结构"是把函数拆开与拼回去的方式（分解与组合）。这条说法在工程上引出两个具体问题。

第一，输入端：多模态接地到底接什么？是模态越多越好，还是按"缺哪个维度补哪个"？

第二，内部结构：能力应该共享一套参数，还是各管各的？共享省参数但互相干扰，隔离不干扰但参数翻倍——中间有没有更好的点？

本篇各做一个玩具实验。

二、实验一：多模态接地的残差底

世界有四个隐藏维度：形状（2 类，与目标无关的干扰项）、颜色（6 个色阶）、材质（3 类）、味道（4 类）。目标要求精确组合：y = 颜色 × 12 + 材质 × 4 + 味道，共 72 类。

各模态能看到什么：文本给形状与粗色（颜色只分两档）；视觉给精确颜色；触觉给材质；味觉给味道。另设一个冗余视觉模态，与视觉重复。

方法：先枚举法算每种模态组合的贝叶斯最优误差（残差底），再训练一个小网络用同样的输入，看它能否达到这个底。

结果：网络在每种组合上都达到了贝叶斯底。

- 仅文本：0.9722；

- 文本 + 视觉：0.9167；文本 + 触觉：0.9167；文本 + 味觉：0.8889；

- 文本 + 视觉 + 触觉：0.7500；

- 文本 + 视觉 + 触觉 + 味觉：0.0000；

- 文本 + 冗余视觉：0.9167（与只有视觉时完全一样）。

解读：残差底由"覆盖了哪些隐藏维度"决定，不是由模态数量决定。每补上一个缺失维度，底就降一截；补全就到底。冗余模态不降底，因为它的信息已经在别处。工程含义很直接：接哪种模态，应该看目标依赖哪些维度、哪些维度还没被覆盖，而不是笼统地追求模态数量。

三、实验二：组合律显式化

四个任务：奇偶校验（8 位）、层级规则、4 位加法、多数投票。输入统一补齐到 10 维。

三种结构。单体：共享主干加四个任务头，联合训练。模块化：每个任务一个独立网络。路由：共享主干，每个任务一个适配器，任务编号已知（相当于完美的路由）。

流程分两阶段。第一阶段只训前三个任务。第二阶段只给第四个任务的数据，旧数据不再出现——这是持续学习的标准设置。

结果：

- 单体：第一阶段旧任务平均 0.809（其中奇偶只有 0.574，早期就互相干扰）；第二阶段旧任务掉到 0.515（加法从 1.00 掉到 0.29），掉了 0.294——灾难性遗忘；新任务 1.00。

- 模块化：旧任务 0.998，第二阶段仍是 0.998，完全不掉；新任务 1.00。

- 路由：旧任务平均 0.893，第二阶段仍是 0.893，不掉；新任务 0.994。但奇偶只有 0.725，说明共享主干仍有干扰。

- 参照：单体用全部四个任务的数据联合重训（需要旧数据），奇偶 0.822、规则 1.00、加法 0.871、多数 1.00。

解读：共享参数换来样本效率，但代价是干扰与遗忘；隔离参数换来稳定，但失去共享。路由是折中：新任务可以单独加、旧任务不掉，但共享主干仍会把干扰留在那里。

四、合起来看

第一个实验说：输入端要按"残差底"配置——缺哪个维度补哪个，冗余模态不解决问题。

第二个实验说：内部结构要在"共享"与"隔离"之间做取舍——显式组合律（模块、路由）换来可扩展与抗遗忘，代价是共享性；共享单体换来参数效率，代价是干扰与遗忘。

这与理论里的说法一致：结构就是分解与组合的方式；组合律越显式，系统越容易扩展，但共享的好处也越少。工程上的含义是：先算清楚"哪些能力互相干扰"，再决定哪些共享、哪些隔离——而不是一律共享或一律模块化。

五、局限

第一，两个实验都是合成任务与玩具模型，和真实大模型不是一回事。

第二，多模态实验里的"贝叶斯底"是精确可算的，这只有在合成世界才成立；真实任务只能估。

第三，组合实验里的路由是"完美路由"（任务编号已知），真实路由要自己学会，会出错。

第四，遗忘实验只测了一轮，长期多轮漂移下的行为没测。

六、可复现

实验一：993_multimodal_residual_v2.py（枚举算底 + 小网络验证）。实验二：994_composition_modular.py（单体/模块化/路由的主对照）、996_full_E.py（路由的调参版）。依赖 PyTorch（Windows 上经 DirectML 跑 AMD GPU）。

附：标注

- **[事实]** 实验一各组合的残差底与网络误差（两者一致）；实验二各架构的两阶段准确率与掉点。

- **[解释]** "残差底由覆盖的维度决定""显式组合律换可扩展性"——对数字的解读。

- **[推测]** 对真实大模型"接地按残差底配置、组合律显式化"的建议——玩具结论的推广，未在真实模型上验证。

- **[不声称]** 不声称提出新的多模态或持续学习算法；模块化、路由、回放都已有大量工作。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第十七篇 · 按计算厚度分配算力：预算与动员

> 单独 DOI：https://doi.org/10.5281/zenodo.23612112

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 实验篇 11（内部编号第 23 篇）

> 日期：2026-10-25

摘要

本篇测"计算厚度"在工程上的第一个后果：算力该给多少、给谁。计算厚度指的是组合律在时间轴上依次应用的次数（时序组合的长度）——它是"路有多长"，不是"路口有多宽"（后者是分布厚度，另篇再谈）；它不是函数或函数类，而是一个长度。

两个实验。

实验一（预算）：世界是一个需要 k 步迭代才能到目标的映射，任务按厚度分四档（k = 1、2、4、8）。三种策略：所有任务都走 8 步、都走 4 步、按厚度路由（任务 k 走 k 步）。结果：路由只用 47% 的算力，平均误差 0.0812，与均匀 8 步（0.0834）相当甚至略好；均匀 4 步（0.0881）反而不如路由。过度计算也有害：k=1 的任务，1 步误差 0.0014，2 步 0.0160，4 步 0.0205。

实验二（动员）：四个不同世界规则，厚度同样是 1、2、4、8 步。模型是"常驻底座（一次变换，所有任务共享）加可动员循环块（按任务厚度走若干步）"。结果：均匀 8 步彻底失败（平均误差 1.756）——对 k=1 的任务，多走的步数直接算过了目标；按需动员 0.434（15 步）；各自独立 0.064（同样 15 步，但每任务一个网络）。

两条结论：第一，计算厚度必须显式匹配，均匀深度会把薄任务算坏；第二，常驻共享加按需动员能工作，但共享的动员块仍有干扰，冲突大的任务应该分开。都是玩具实验。

一、问题

我们把"厚度"分成两种：分布厚度（同一输入对应多宽的一族输出）和计算厚度（从输入走到目标要多少步）。本篇只动计算厚度：任务需要 k 步就是 k 步，答案唯一（分布厚度为零）。

在展开实验之前，要把"计算厚度"说得更精确一点。

组合律（把切片拼回函数的方式）有两种用法。一种是沿时间轴依次应用：第一步的输出喂给第二步，第二步再喂给第三步——这叫时序组合。另一种是同时发生：并行、共享状态、路由选择——这叫并列组合。计算厚度只量前者：**在时间轴上要依次应用几次组合，才能从输入走到目标。**

所以有两个函数空间要分清。

第一个是切片函数空间：被组合的对象，每一点是一个静态函数（一层、一块）。

第二个是组合律空间：起组合作用的高阶函数空间，每一点是一种"把切片拼成函数"的方式。它沿时间展开成 C^1、C^2、……——一步组合、两步组合、三步组合。

计算厚度不是函数，也不是函数类的成员，而是组合律空间里的一条最短路径长度：目标函数第一次落进某个 C^k 的像时，那个 k 就是它的计算厚度。它由目标、可用原语、组合律三者共同决定：原语越粗（一步能顶多步），同一个目标的厚度越小。

问题有两个：算力该给多少？共享还是分开？

二、实验一：算力预算

世界：x_{t+1} = tanh(A x_t)，A 是固定的随机矩阵，状态 8 维。任务：给初值 x0，预测 x_k，k = 1、2、4、8。

模型：共享循环细胞，每步 h ← tanh(W h + U x0 + b)，最后线性读出；步数就是计算量。

三种策略：均匀深（都走 8 步）、均匀浅（都走 4 步）、按厚度路由（任务 k 走 k 步）。

结果（每任务取不同宽度的最好）：

| 策略 | 平均误差 | 总步数 |

|---|---|---|

| 均匀 8 步 | 0.0834 | 32 |

| 均匀 4 步 | 0.0881 | 16 |

| 按厚度路由 | 0.0812 | 15 |

路由用 47% 的算力，误差与均匀 8 步差 0.0022，实际还略低。

过度计算单独测：k=1 时，1 步误差 0.0014，2 步 0.0160，4 步 0.0205，8 步 0.0148。多算不更好，单独看反而更差——世界是迭代映射，多走一步就过了目标。

解读：算力应该按"任务需要多少步"分配，而不是统一给一个深度；均匀深度的浪费可以测出来，约一半。

三、实验二：常驻底座与按需动员

人想难问题时不只是"想更久"，还会动员更多结构：有的回路长，有的短，有的常驻（维持、注意）。本篇把这个图像做成一个玩具结构。

四个任务，每个任务有自己的世界规则：任务 i 是 x_{t+1} = tanh(A_i x_t)，给 x0 预测 f_i^{k}(x0)，k = 1、2、4、8。

模型：常驻底座（一个线性层加 tanh，所有任务共享，走一次）加可动员循环块（共享权重，按任务厚度再走若干步），最后线性读出。

对照：均匀深（都走 8 步）、按需动员（任务 k 走 k 步）、均匀浅（都走 4 步）、各自独立（每任务一个网络，总步数与动员相同）。

| 策略 | 各任务误差（k=1,2,4,8） | 总步数 | 平均误差 |

|---|---|---|---|

| 均匀 8 步 | 1.885 / 1.853 / 1.706 / 1.579 | 32 | 1.756 |

| 按需动员 | 0.242 / 0.439 / 0.560 / 0.496 | 15 | 0.434 |

| 均匀 4 步 | 1.905 / 1.842 / 1.717 / 1.571 | 16 | 1.757 |

| 各自独立 | 0.004 / 0.019 / 0.103 / 0.130 | 15 | 0.064 |

三点解读。

第一，均匀深严重失败：对 k=1 的任务，目标在一步之后；模型硬走 8 步，等于反复迭代，算到了别的地方。步数越多不越好，甚至越差——这与实验一的过度计算一致。

第二，常驻底座加按需动员能工作：平均误差 0.43，比均匀策略好一个量级，步数只有一半不到。

第三，各自独立最好（0.06）：不是因为步数更少（同样 15 步），而是因为每个任务有自己的循环块，不受其他任务干扰。共享循环块在不同世界规则之间产生了干扰。

三点五、人脑层面的对应：接论文**-36**（第五篇；原论文**-21** 已并入）

人脑层面的解释不需要在这里重新展开。论文-36《语言与大脑：两种函数形式》（其前身论文-21《人脑的可函数化形式》已并入）已经把脑写成可函数化器官：表示、层级、内生时间、有限自指；并且指出它是混合型、结构异质——有常驻的结构（维持、注意），也有按任务临时动员的长短回路，而它本身不自足、被保住的是"身体 + 脑 + 环境"的整体。

本篇的"计算厚度"与"按需动员"，就是那个器官层面的工程投影：简单的任务少动员，复杂的任务多动员；常驻结构一直用，临时结构按需用。细节以论文-36 为准，这里只作为设计直觉，不作为本篇的论据。

四、合起来看## 四、合起来看

实验一说：算力按计算厚度分配；均匀分配浪费近一半，过度计算会算坏。

实验二说：常驻底座共享是合理的（省参数、保留共同表示），但动员部分共享有干扰；冲突大时应分开。

对应的设计问题：哪些结构常驻共享，哪些按任务动员——这正是"共享还是分离"的取舍，和前面残差底与组合律实验的结论一致。

五、局限

两个实验都是合成世界与玩具模型。动员实验里"任务需要几步"是给定的（完美动员），真实系统要自己估计。独立方案虽好但参数翻倍，没测"独立 + 少量共享"的折中。实验二只跑了一次运行，没有多种子复核。

六、可复现

实验一：1000_thickness_budget.py。实验二：1010_recruit.py。依赖 PyTorch（Windows 上经 DirectML 跑 AMD GPU）。

附：标注

- **[事实]** 两个实验的数字：路由 47% 算力、误差 0.0812；过度计算更差；均匀 8 步 1.756、动员 0.434、独立 0.064。

- **[解释]** "算力按厚度分配""常驻共享、动员分离"——对数字的解读。

- **[推测]** 人脑"常驻结构 + 长短结构"的类比——方向一致，没有证据链。

- **[不声称]** 不声称提出新的算力调度或网络架构；早退、门控、模块化已有大量工作。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第十八篇 · 层厚放在哪：异质层实验

> 单独 DOI：https://doi.org/10.5281/zenodo.23612126

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 实验篇 12（内部编号第 24 篇）

> 日期：2026-10-25

摘要

大模型的层是均匀的：每层同宽、同结构，每个输入走同样深度。生物不是：不同区域的结构不同，有的回路短，有的长，有的常驻。

本篇问一个很窄的问题：同样的参数与算力，"层厚"放在网络的不同位置，结果一样吗？

任务：四个多任务（奇偶、层级规则、加法、多数），共享主干加各自输出头。四种结构：均匀 6×64（前向乘加约 21k）、宽前 128→16（约 15k）、宽后 16→128（约 14k）、中间厚（约 17k）。

三个种子的复核结果：宽后 16→128 平均准确率 0.995，用 14k 算力、19k 参数；均匀 6×64 是 0.980，用 21k 算力、24k 参数；中间厚 0.954；宽前 0.831。宽后分任务看提升主要来自难任务（奇偶 0.994 对 0.981，加法 0.985 对 0.941）。

结论：层厚放在哪里，比"每层都多厚"更重要。让前面几层窄（先把输入压成表示）、后面几层宽（再展开成答案），在这组任务上更省也更准；反过来（宽前）明显差。这与生物分层的异质方向一致，但我们只测了一种任务组合、几个结构，不能推广成定律。

一、问题

前一篇测了"算力给多少"（时间方向）。本篇测另一个方向：层与层之间的厚度分布（空间方向）。

大模型是均匀层厚；人脑是异质的：区域之间的结构、连接、时间常数都不同。如果厚度分布重要，那么同参数同算力下，"放在哪里"应该 measurable 地影响结果。

二、设定

任务：四个多任务——奇偶校验（8 位）、层级规则、4 位加法、多数投票。输入统一补齐到 10 维；共享主干，每个任务一个输出头，训练时任务混合。

四种结构（宽度序列）：

- 均匀：6 层 × 宽 64；

- 宽前：128 → 64 → 48 → 32 → 24 → 16；

- 宽后：16 → 24 → 32 → 48 → 64 → 128；

- 中间厚：[16, 32, 64, 96, 64, 32]。

算力用前向乘加数衡量（输入维到第一层 + 层间乘加），分别约 21k、15k、14k、17k；参数量分别约 24k、16k、19k、19k。

训练：任务混合，每步随机抽一个任务；6000 步。宽后与均匀做了三个种子，其余各一次。

三、结果

| 架构 | 前向乘加 | 参数 | 平均准确率（三种子） |

|---|---|---|---|

| 均匀 6×64 | 约 21k | 24k | 0.980（0.969 / 0.985 / 0.988） |

| 宽后 16→128 | 约 14k | 19k | 0.995（0.993 / 0.993 / 0.998） |

| 中间厚 | 约 17k | 19k | 0.954（一次） |

| 宽前 128→16 | 约 15k | 16k | 0.831（一次） |

宽后结构用约 67% 的算力、更少的参数，平均准确率反而更高。分任务看，提升主要来自难任务：奇偶 0.994 对 0.981，加法 0.985 对 0.941；规则与多数两个任务在所有结构上都是满分（太简单，分不出差别）。

宽前结构明显差（0.831），中间厚居中（0.954）。

四、解读

第一，层厚分布是可测量的：同样的算力，位置不同结果差 0.16（0.995 对 0.831）。

第二，在这组任务上，"前窄后宽"最划算：前面几层把输入压成较窄的表示，后面几层再展开到答案。一种可能的解释是：窄层先做选择（丢掉无关信息），宽层再组合（难任务需要更宽的表示），顺序对了就省。

第三，这与生物分层的异质方向一致——不同区域结构不同，不是"同一种砖摞高"。但我们只测了四个任务、四种结构，还不能说"前窄后宽"是通则。

四之二、人脑层面的对应：接论文**-36**（第五篇；原论文**-21** 已并入）

人脑层面的解释也不需要在这里重写。论文-36《语言与大脑：两种函数形式》（其前身论文-21《人脑的可函数化形式》已并入）已经把脑写成可函数化器官，并指出它是结构异质的：不同区域的结构、连接、时间常数不同，有常驻的部分，也有按任务动员的长短回路。

本篇测的"层厚放在哪"，正是这种异质性的一个最粗的工程投影：不是所有层都该一样厚；把厚度放到哪一段，结果会不一样。论文-36 给出的定性图像（异质、常驻、动员）在这里变成一个可测的问题（同算力下，宽前/宽后/均匀/中间厚哪个好）。

同样，细节以论文-36 为准；本篇只把它当设计直觉，不作论据。

五、局限

任务与模型都是玩具规模；宽前与中间厚只跑了一次运行，种子起伏可能影响结论；宽后与均匀做了三个种子。任务里的规则与多数太简单（全满分），主要区分力来自奇偶与加法。真实大模型的层不是全连接（有注意力、残差、归一化），这里的"层宽"只是最粗的近似。

六、可复现

脚本：1007_heterogeneous_layers.py（主对照）、1008_hetero_fair.py（同算力对照）、1009_hetero_seeds.py（三种子复核）。依赖 PyTorch（Windows 上经 DirectML 跑 AMD GPU）。

附：标注

- **[事实]** 四种结构的算力、参数、各任务准确率；宽后 0.995 vs 均匀 0.980，宽前 0.831。

- **[解释]** "层厚放在哪里更重要""前窄后宽更省"——对数字的解读。

- **[推测]** 与生物异质分层的类比——方向一致，没有证据链。

- **[不声称]** 不声称提出新的网络架构；宽度变化、层剪枝、条件计算都已有大量工作。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第十九篇 · **RLHF** 压平人类分歧

单独 DOI：https://doi.org/10.5281/zenodo.23611877

何淇 · 与 DeepSeek 协作

数字生命系列 · 实验篇 13（内部编号第 25 篇）

日期：2026-10-25

摘要

本篇测"分布厚度"在工程上的后果：对齐会把宽的输出分布压平。

设定沿用前面的人类分歧实验：题目分 thin（事实题，人类答案一致）与 thick（判断题，答案取决于偏好，人类有分歧）。基础模型学 P(答案|题目)，训练后冻结；thick 题上它的输出熵 0.9655，很接近人类的 1.0138——也就是说，基础模型是诚实地保留分歧的。

然后做偏好微调的玩具版：对每题从当前模型采样两个答案，让标注者按人类偏好选一个，再在选中的答案上做一步训练。

结果：thick 题的人类答案熵不变（1.0138），但模型熵从 0.9655 降到 0.4090（下降约 57%）；与人类分布的 KL 从 0.0451 涨到 0.4188（约 9 倍）；与"人类多数答案"的一致率从 0.870 升到 0.935。thin 题人类熵约为零，模型变化很小。

解读：人类的分歧没有错，但偏好微调把模型的输出分布压到了多数答案上。模型变得坚决，也更偏离人类的真实分布——少数意见被丢掉了。这是"厚的地方压不平"在工程上的具体后果：对齐把分布厚度压成确定性，代价是失去分布。玩具实验，不涉及真实大模型。

一、问题

我们把厚度分成两种：计算厚度（需要多少步）与分布厚度（输出分布有多宽）。前一篇测计算厚度；本篇只动分布厚度：题目都是单步判断（计算厚度小），变化的只是输出有多宽——人类答案熵从 0（thin）到约 1.01（thick）。

问题：对齐（偏好微调）对分布厚度做了什么？

二、设定

模拟 60 个人类，每人有一个偏好向量。thin 题：答案由输入决定，人人一致。thick 题：答案取决于偏好，人与人不同。人类答案分布的熵用来量厚度。

基础模型：小 MLP 学 P(答案|题目)；训练数据是每题抽一个人来答。因为见过很多人的答案，它学到的是人类总体分布。

偏好微调（玩具版）：对每题从当前模型采样两个答案，按人类偏好选一个（人类偏好概率高的胜出），在选中的答案上做硬标签训练一步；重复多轮。

三、结果

| 指标（thick 题） | 人类 | 基础模型 | 偏好微调后 |

|---|---|---|---|

| 答案熵 | 1.0138 | 0.9655 | 0.4090 |

| 与人类分布 KL | — | 0.0451 | 0.4188 |

| 与多数答案一致率 | — | 0.870 | 0.935 |

thin 题：人类熵约为 0，基础模型 0.1413，偏好微调后 0.1142（变化很小）。

四、解读

第一，压平是真实发生的：模型熵下降约 57%，而人类的分布没有变。

第二，压平不全是"学得更准"：与多数答案的一致率确实升了（0.870 到 0.935），但那是把少数意见的概率让给了多数答案——与人类真实分布的 KL 涨了近 9 倍。

第三，thin 题几乎不受影响：那里本来就没有分歧可压。

用我们的词说：偏好微调在做"单峰化"——把一条宽的分布压成一个点。对事实题这是好事（本来就是尖的）；对判断题这是丢失：丢掉的是"人本来就有不同意见"这件事。

五、局限

第一，这是玩具：标注者是模拟的（按人类偏好分布选），不是真人；真实 RLHF 还有 KL 正则项，压平会被约束（本篇没加）。

第二，题目是合成的（thin/thick 两类），人类偏好是程序生成的。

第三，只测了一轮偏好微调，没测多轮下的长期漂移。

六、可复现

脚本：1002_rlhf_flatten.py（软标签版，作对照）、1003_rlhf_flatten_v2.py（偏好采样版，本文数据来自这个）。依赖 Python 3 与 numpy。

附：标注

- **[事实]** thick 题：人类熵 1.0138、基础模型 0.9655、微调后 0.4090；KL 0.0451→0.4188；一致率 0.870→0.935。

- **[解释]** "对齐压平分布、丢掉少数意见"——对数字的解读。

- **[推测]** 真实 RLHF 会因 KL 正则而减轻压平——未验证。

- **[不声称]** 不声称提出新的对齐算法；KL 正则、多样性保留都已有大量工作。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第二十篇 · 空间、组合与厚度：三个资源的交换实验

> 单独 DOI：https://doi.org/10.5281/zenodo.23629197

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 实验篇 14（内部编号第 27 篇）

> 日期：2026-10-25

摘要

前面几篇把"可表示"拆成三个资源：空间上的表示维度（宽度 S）、组合律上的表示维度（组合自由度 C）、时间上的步数（计算厚度 T）。本篇做三个实验，测这三种资源各自管什么、能不能互换。

实验一（三因子）：在空间型、组合型、厚度型任务上扫 (S, C, T)。结果：空间型任务由 S 决定（S=32 以上满分，T、C 无影响）；厚度型任务需要 S 和 T 都够（k=4 时 S=32：T=1 误差 0.185、T=4 降到 0.077）；组合型任务必须用"输入相关路由"才算组合维度——只并联相加时 C 没有作用，带路由后 C=2 在路由任务上明显有效（k=4 时 C=2、T=4 误差 0.146，C=1 同配置 0.222）。

实验二（S×T 平面）：扫 90 个配置画等误差曲线。结果：交换率不是常数——宽度比步数更值钱（k=4 误差≈0.12 时，S=64、T=3 与 S=32、T=4 相当，但 S×T 并不守恒）；步数不够有断崖（T=1 时 k=8 误差 0.468，加宽到 S=128 也只能降到 0.37）；过度计算有害（T=6→8 多数曲线回升）。图 1 是三个厚度下的曲线。

实验三（层 vs 循环）：同一个迭代任务，比较"T 个不同层"与"同一层循环 T 次"。结果：两者精度几乎相同（k=4、T=4 时：不同层 0.059，循环 0.072），但循环的参数少得多（T=8 时 1096 对 30216，约 27 倍）。也就是说，计算上"加层"和"循环"等价，差别在参数共享方式与语义。

结论：三个资源各有分工，可部分互换但汇率有偏向；循环是参数效率更高的"时间模拟"，不是唯一能实现时间的手段。人脑的"按需收敛"（一次结构试探、没收敛再循环、收敛后固化）与这些结论方向一致，但本篇不声称解释人脑。

一、问题：三个资源

我们前面把"可表示"拆成三个位置：

- 空间上的表示维度：切片内部的轴数，实验里用宽度 S 控制（单隐层 rank 上界 ≈ S）；

- 组合律上的表示维度：组合方式的自由度，实验里用分支数 C 与是否路由控制；

- 时间上的步数：迭代次数 T。按我们的口径，时间不叫"维度"，叫步数。

事先的预测（预计算版设计文档）：

| 任务 | 决定资源 | 预测边界 |

|---|---|---|

| 空间型 | S | 沿 S 的竖线 |

| 组合型 | C | 沿 C 的竖线 |

| 厚度型 | T（S 部分可换） | 斜线 |

| 空间+厚度 | S 和 T | L 形 |

| 组合+厚度 | C 和 T | L 形 |

本篇测试这些预测。

二、实验一：三因子

模型：C 个并行分支（各自宽 S、迭代 T 步），合并后读出。任务：

- 空间型：6 位输入到 64 类的任意查表函数；

- 组合型：两路组合（奇偶 XOR 多数），以及后面的路由型任务；

- 厚度型：x_{t+1}=tanh(A x_t)，预测 x_k（k=1、4）。

空间型：S=8 时准确率 0.96~0.98，S=32/64 全部 1.000；T 和 C 改变没有影响。预测"沿 S 的竖线"成立。

厚度型（k=4）：S=8 全失败（误差 0.32~0.43）；S=32：T=1 误差 0.185、T=4 0.077、T=8 0.094；S=64：T=1 0.138、T=4 0.056、T=8 0.063。需要 S 和 T 都够，且 T 过多变差——L 形趋势出现，过度计算有害。

组合型（路由任务）：先试"并联相加"的 C 分支（无路由），C 没有任何优势——说明把多个分支加起来不等于组合维度。改成输入相关路由（softmax 选分支）后：路由+k=1 时，S=32、C=2 在 T=1 就达到 0.046（C=1 同配置 0.055）；路由+k=4 时最优 S=64、C=2、T=4 误差 0.146，对比 C=1、T=4 的 0.222（降 34%），而 C=2、T=1 是 0.443。组合维度真实有效，但必须是"输入相关选路"，并且和厚度有交互（L 形）。

三、实验二：**S×T** 平面与交换率

网格：S ∈ {8,16,32,64,128} × T ∈ {1,2,3,4,6,8}，三个厚度 k=2、4、8，共 90 个配置（6 进程并行，约五分钟）。图 1 是结果曲线。

读图与数据：

第一，宽度决定误差能降到多低。k=8 时 S=8 一直停在 0.6 左右，S=128 能到 0.17。

第二，步数决定降得多快。所有曲线在 T=1 到 T=2 之间大幅下降，之后趋于平坦。T 不够有断崖：k=8、T=1 时误差 0.468，即使 S=128 也只能到 0.37。

第三，交换率不是常数。若按 S×T ≈ 常数，S=64、T=4 与 S=32、T=8 应该接近；实测前者 0.056、后者 0.094。在等误差点上，宽度比步数更值钱：k=4 误差≈0.12 时，(S=64, T=3) 与 (S=32, T=4) 相当。也就是说，"时间换维度"存在，但汇率偏向空间——不是一维换固定几步。

第四，过度计算有害。T 超过 4~6 后误差回升：k=2 时 T=8 的 0.045 差于 T=3 的 0.042；k=4、S=32 时 T=8 的 0.125 差于 T=6 的 0.107；k=8、S=128 时 T=8 的 0.180 差于 T=6 的 0.170。

四、实验三：层 **vs** 循环

同一个迭代任务（x_k，k=2、4、8），两种模型：T 个不同层（结构差异），或同一层循环 T 次（时间）。

结果（S=64）：

| k=4 | T=1 | T=2 | T=3 | T=4 | T=6 | T=8 |

|---|---|---|---|---|---|---|

| 不同层 | 0.349 | 0.152 | 0.103 | 0.059 | 0.052 | 0.064 |

| 循环 | 0.349 | 0.152 | 0.097 | 0.072 | 0.065 | 0.070 |

| k=8 | T=1 | T=2 | T=3 | T=4 | T=6 | T=8 |

|---|---|---|---|---|---|---|

| 不同层 | 0.496 | 0.325 | 0.251 | 0.193 | 0.154 | 0.165 |

| 循环 | 0.502 | 0.325 | 0.231 | 0.198 | 0.174 | 0.177 |

两者精度几乎相同，不同层甚至略好。但参数：T=8 时循环 1096，不同层 30216，约 27 倍差距。

解读：计算上"加层"和"循环"是同一件事——都是多走几步；差别在参数共享：循环用一套参数反复用，层用每层独立参数。所以"循环才是时间"这句话要修正为：循环是参数效率更高、语义上对应"同一规则沿时间反复"的时间实现；层可以模拟出时间步，代价是更多参数。

五、解读

第一，三个资源各有分工：空间决定"能装多复杂的一步"；组合决定"能选多少种接法"；步数决定"能展开多长"。互换了，但汇率有偏向，且任务需要哪种资源，边界就在哪个方向。

第二，过度计算有害这一点在三个实验里都出现（三因子、S×T 平面、层/循环），与论文-23 的结论一致：步数不是越多越好，超过任务厚度会走到别处。

第三，和脑的类比：一次结构运算试探，没收敛就带记忆循环，收敛了就固化并缩小范围（按需收敛）。层/循环对照说明这条类比在计算上是可行的——循环省参数，生物用时间换空间是合理的；但这只是方向一致，不是证据。

六、局限

第一，全部是合成任务与玩具模型；边界形状依赖具体任务族。

第二，组合维度的结论依赖路由实现：只并联相加时 C 无效，带路由才有效，说明"组合维度"的工程含义要落到具体机制上。

第三，S×T 平面的绝对误差受模型架构细节影响（本实验与三因子实验的模型略有差异，故看趋势不看绝对值）。

第四，层/循环只测了一个任务族（迭代映射）和一个宽度（S=64）。

第五，交换率只在三个厚度上测，且 T 的网格不完全均匀。

七、可复现

三因子：1040_three_factors.py、1041_three_factors_composition.py、1042_three_factors_router.py。S×T 平面：1043_plane_worker.py（多进程分片）。层/循环：1044_layers_vs_loops.py。画图：1050_plot_plane.py（纯 Python 生成 SVG，Edge 转 PNG）。图 1 见同目录 `图-SxT平面曲线.png`。依赖 PyTorch（Windows 经 DirectML 跑 AMD GPU）。

附：标注

- **[事实]** 三个实验的数字与图 1 的曲线：S 决定空间型、S+T 决定厚度型、路由 C 有效、交换率非常数、过度计算有害、层与循环精度相当而参数差 27 倍。

- **[解释]** "三个资源各有分工""循环是参数效率更高的时间模拟"——对数字的解读。

- **[推测]** 人脑按需收敛的类比——方向一致，无证据链。

- **[不声称]** 不声称提出新架构或新算法；门控、路由、循环网络、早退都已有大量工作。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第二十一篇 · 层的路径与冗余：**GPT-2** 上的层扰动实验

> 单独 DOI：https://doi.org/10.5281/zenodo.23663369

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 实验篇 14（内部编号第 33 篇）

> 日期：2026-10-27

摘要

本篇在真实小模型上测一个问题：大模型里的结构是"压在某一层里"，还是"跨几层展开、带冗余"？

做法：在 GPT-2 small（12 层）上，把某一层"跳过"（输出等于输入，保留残差流），分别测单层、相邻两层、连续三层的扰动效果；语料为 tinyshakespeare，300 条 64-token 序列，9600 个预测位置；指标是下一 token 的 top-1 翻转率与损失增加，并按下一个词的类别（数字/标点/功能词/情绪词/身体词/其他）分组。

结果：单层跳过第 1~10 层，top-1 翻转率约 0.20~0.31；跳过最后一层为 0.55；跳过第 0 层为 0.96（异常，需分开看）。相邻两层一起跳过，翻转率升到约 0.37~0.68，比"两个单层中较大的那个"还高约 0.10~0.17——这是冗余的签名：单独缺一层有人补，两层一起缺就没人补了。连续三层跳到约 0.44~0.77。类别之间差异不大，身体词与情绪词在中层略敏感。

结论：GPT-2 的结构是跨层分布且带冗余的。单层扰动会低估结构的重要性（用户提出的"去掉单层只影响完整性"得到支持）；按层做"判定"或"早退"时，判据应当看路径而不是单层。所有结果来自小模型与玩具语料，不声称适用于大模型。

一、问题

前面的框架说：结构是切片 + 组合律；复杂结构要靠时间上的步数展开。如果这条在真实模型里也成立，那么：

1. 一个概念/结构不应只对应某一层，而应跨几层展开；

2. 层与层之间可能有冗余：去掉一层，别的层能补；

3. 因此"单层扰动"会低估结构的重要性。

用户的直觉是：**单层之间也有结构，多层之间也有结构；去掉单层可能只影响完整性，因为有冗余。**本篇就是去测它。

二、方法

- **模型**：GPT-2 small（12 层、宽度 768、124M 参数），DirectML GPU。

- **语料**：tinyshakespeare，随机取 300 条、每条 64 token；预测位置为每个 token（间隔 2），共 9600 个。

- **扰动**：把某些层"跳过"——该层输出 = 输入（恒等直通），保留残差流，只去掉这一层的贡献。这样才是"少算一层"，而不是把信息清空。

- **配置**：单层 12 个；相邻两层 11 个；连续三层 10 个。

- **指标**：相对基线的 top-1 翻转率、损失增加。

- **分组**：按"下一个 token"的类别分组：数字、标点、功能词、情绪词、身体词、其他。

三、结果

单层跳过（top-1 翻转率，按类别）：

层	数字	标点	功能词	情绪	身体	其他	均值
0	0.986	0.833	0.974	1.000	1.000	0.978	0.962
1	0.214	0.185	0.195	0.167	0.271	0.199	0.205
2	0.209	0.195	0.223	0.278	0.333	0.231	0.245
3	0.245	0.214	0.252	0.208	0.312	0.252	0.247
4	0.250	0.199	0.213	0.208	0.292	0.231	0.232
5	0.291	0.198	0.221	0.250	0.375	0.239	0.262
6	0.218	0.188	0.217	0.250	0.292	0.236	0.233
7	0.255	0.195	0.253	0.250	0.229	0.239	0.237
8	0.277	0.199	0.228	0.264	0.396	0.230	0.266
9	0.268	0.167	0.255	0.264	0.396	0.236	0.264
10	0.355	0.222	0.263	0.319	0.438	0.277	0.312
11	0.482	0.510	0.517	0.611	0.708	0.479	0.551

- 第 0 层跳过几乎全崩（0.96）：第一层被跳过时整条链的输入分布就变了，需单独说明（见局限）。

- 第 1~10 层：单层跳过只翻 20%~31%，说明**多数单层不是致命的**。

- 最后一层（11）：0.55，明显更重要——输出前的变换不能省。

相邻两层跳过（组翻转率 vs 单层最大）：

| 组 | 组均值 | 单层最大均值 | 冗余量 |

|---|---|---|---|

| (1,2) | 0.400 | 0.246 | 0.155 |

| (2,3) | 0.423 | 0.262 | 0.160 |

| (3,4) | 0.417 | 0.248 | 0.169 |

| (4,5) | 0.387 | 0.263 | 0.125 |

| (5,6) | 0.368 | 0.262 | 0.106 |

| (6,7) | 0.375 | 0.247 | 0.128 |

| (7,8) | 0.387 | 0.271 | 0.116 |

| (8,9) | 0.412 | 0.271 | 0.141 |

| (9,10) | 0.457 | 0.312 | 0.145 |

| (10,11) | 0.681 | 0.551 | 0.130 |

- 每对相邻层都出现正的冗余量（约 +0.10~+0.17）。

- 含义：**单独缺一层，别处能补；两层一起缺，就没人补了。这正是"冗余"的结构签名。**

连续三层跳过：

| 组 | 各类别翻转范围 | 均值 |

|---|---|---|

| (1,2,3) | 0.475~0.729 | 0.584 |

| (3,4,5) | 0.446~0.625 | 0.516 |

| (4,5,6) | 0.382~0.514 | 0.436 |

| (6,7,8) | 0.444~0.562 | 0.498 |

| (9,10,11) | 0.670~0.875 | 0.774 |

- 三层一起跳，翻转率到 0.44~0.77，明显高于任意单层。

- 结构不是"压在一层"，是**跨层展开**的。

类别差异： 多数层内类别差异在 0.1 以内；身体词、情绪词在中层略更敏感（例如第 5/8/10 层身体词 0.375~0.438）。这说明概念对层的依赖有一点选择性，但不像"专用层"那样干净。

四、解释

1. **结构跨层展开**：单层扰动温和、成组扰动急剧上升——一个结构由几层共同完成，缺一层还能靠其余层部分完成。

2. **冗余**：相邻层的正冗余量（+0.10~+0.17）说明层与层之间存在部分可替代性。用户说的"去掉单层只影响完整性"在数据上得到支持。

3. **单层 ablation 的低估**：只看单层会得出"这些层都不重要"的错误印象；真实结构的重要性藏在路径与组合里。

4. **按路径判定**：如果要做判定器/早退，判据应看"这段路径够不够"，而不是"这一层够不够"。

五、可检验推论

1. 冗余量应随模型规模、训练分布变化：在更偏向"必须逐步计算"的任务上，冗余应减少。

2. 有冗余的相邻层，应该可以用"共享参数的循环"部分替代而不掉点——这正好接上"循环比加层省参数"的已有结果。

3. 概念选择性应表现为"路径差"而不是"单层差"：不同概念的关键层组合应不同。

4. 若在真实大模型上重复，单层扰动应同样低估重要性；成组扰动的冗余签名应依然出现。

六、局限

第一，只有 GPT-2 small 与 tinyshakespeare，规模小、语料窄，不能外推到真实大模型。

第二，第 0 层跳过异常（0.96）：跳过第一层等于去掉第一层注意力的输出，整条链的输入分布变化过大；这个数字不宜与其他层并列解释。

第三，"跳过"是推理期扰动，不等于"训练时没有这层"；训练时若少一层，网络会重新分配功能，结论会不同。

第四，类别词表很粗（数字/标点/功能词/情绪/身体/其他），"概念选择性"只测到很粗的粒度；更细的概念（句法结构、语义角色）没有测。

第五，只测了 top-1 翻转与损失，没有测表示空间的变化；冗余也可能表现为"预测不变但内部表示变了"。

第六，没有与"层与循环"的替代实验直接衔接——那是下一步。

附：标注

- **[事实]** 表中所有翻转率与损失来自本次 GPT-2 实验（数据文件：数据-GPT2层路径.json）。

- **[解释]** "结构跨层展开""相邻层冗余""单层扰动低估重要性"——对数据的解读。

- **[推测]** 四条可检验推论——方向性，未验证。

- **[不声称]** 不声称适用于大模型；不声称解释了概念表征；不声称"层冗余"等于"可无代价删层"。

> 本文由何淇创作，DeepSeek（AI）协助整理与写作；观点与责任属于作者。

第二十二篇 · 语言结构作为自变量：**GPT-2** 上的三组实验

> 单独 DOI：https://doi.org/10.5281/zenodo.23665217

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 实验篇 15（内部编号第 34 篇）

> 日期：2026-10-27

摘要

前面的实验（论文-33、以及本篇的早期版本）是从"脑结构"出发设计的：层、循环、判定、维持。但这些机制不是大模型被训练出来的方式——大模型是被语言塑形的。本篇换一个自变量：**语言结构强度**（用完整模型预测的熵作为代理，分低/中/高熵三组），重做三组实验。

第一组，层冗余 × 语言结构：跳过单层或相邻两层，看 top-1 翻转率。结果：低熵（语言可预测）位置的翻转率明显低于高熵位置（单层 0.49~0.50 vs 0.76~0.77；两层 0.48~0.53 vs 0.74~0.76）。语言结构强的位置更容忍删层。

第二组，判定器特征对比：同一预算下比较四种 halt 特征——A 置信度、B 跨层路径一致性、L 语言结构特征（词类、位置、重复、首层熵）、A+L。结果：L 在较深的预算下最好（深度 8：0.161 vs A 0.153）；路径特征 B 没有优势。

第三组，按语言结构早退：用 L 特征决定退出，在相同平均深度下比按置信度早退略好（深度 8~10 高出约 0.5~0.8 个点）。

结论：**"语言结构强度"是比"脑机制"更合适的自变量**——语言越可预测，位置越早收敛、越容忍删层、越适合早退；而"内部路径是否稳定"这个脑结构式的判据，在语言层框架下并不带来额外信息。所有结果来自 GPT-2 small 与 tinyshakespeare，不声称适用于大模型。

一、为什么换自变量

前面我们一直把大模型当"一个器官"来测：它有几层、能不能循环、何时收敛、能不能早退。这些问法来自对大脑结构的类比。但大模型不是被脑机制训练的，它是被语言训练的——语言的结构（句法、语义、叙事、常识）塑形了它的表示与组合方式。

所以更合适的自变量是**语言结构强度**：一个位置在语言上有多可预测、结构有多强。我们用它重做三组实验，检验一个统一的说法：

> 语言结构强的位置，会更早收敛、更容忍删层、更适合早退。

二、方法

- **模型**：GPT-2 small（12 层、768 宽、124M），DirectML GPU。

- **数据**：tinyshakespeare，300 条 64-token 序列，取预测位置（间隔 2），共 9600 个。

- **语言结构强度**：用完整模型在该位置的下一个词分布熵作为代理，取三分位分三组（低熵=可预测、中熵、高熵=不可预测）。

- **实验 A（层冗余）**：跳过单层（2/6/10）与相邻两层（2-3/6-7/10-11），统计相对基线的 top-1 翻转率，按熵组比较。

- **实验 B（判定器）**：每层训练 halt 判定器，特征四组：A=置信度/熵；B=跨层路径（相邻层一致、KL、趋势、cos 变化）；L=语言结构（词类、位置、重复、首层熵）；A+L。标签=该层预测是否正确。比同平均深度下的准确率。

- **实验 C（早退）**：用 B 中学到的判定器做退出策略，比较不同特征下的准确率—深度曲线。

三、结果

3.1 实验 A：层跳过 × 语言结构（top-1 翻转率）

| 跳过层 | 低熵（可预测） | 中熵 | 高熵（不可预测） |

|---|---|---|---|

| [2] | 0.491 | 0.694 | 0.771 |

| [6] | 0.502 | 0.690 | 0.763 |

| [10] | 0.498 | 0.689 | 0.763 |

| [2,3] | 0.484 | 0.653 | 0.737 |

| [6,7] | 0.534 | 0.698 | 0.760 |

| [10,11] | 0.697 | 0.911 | 0.957 |

- 所有配置下，低熵组的翻转率都明显低于高熵组（典型：单层 0.49 vs 0.77）。

- 含义：**语言结构强的位置，删掉一两层也不影响预测**——冗余（或语言先验的补位）集中在这里。

- 末两层（10,11）的跳过对所有组都更伤，但低熵组仍更稳。

3.2 实验 **B**：判定器特征对比（同平均深度下的准确率）

| 深度 | A 置信度 | B 路径 | L 语言 | A+L | B+L |

|---|---|---|---|---|---|

| 3 | 0.149 | 0.148 | 0.150 | 0.147 | 0.148 |

| 5 | 0.154 | 0.147 | 0.153 | 0.151 | 0.153 |

| 8 | 0.153 | 0.152 | **0.161** | 0.159 | 0.159 |

| 10 | 0.158 | 0.156 | 0.160 | 0.160 | 0.159 |

- 浅预算（深度 3~5）：各特征基本持平。

- 深预算（深度 8~10）：L（语言结构特征）最好（0.161 vs 0.153），A+L、B+L 次之。

- **B（跨层路径）始终没有优势**——我们早期"判定要看路径"的预测，在语言层框架下依然不被支持。

3.3 实验 **C**：按语言结构早退

- 用 L 特征做退出判定：在相同平均深度（8~10 层）下，准确率比按置信度早退高约 0.5~0.8 个点。

- 与完整模型（12 层，准确率约 0.169）相比，早退到平均 10 层可保住约 0.16（掉不到 1 个点）。

- 与实验 A 一致：语言结构强的位置，退出更安全。

四、解释

1. **语言结构强 ⇒ 早收敛**：可预测的位置，模型不需要深层计算，语言先验几乎决定了答案；不可预测的位置需要更多层，且更难判对。

2. **语言结构强 ⇒ 冗余高**：删层对低熵位置影响小，因为其他地方（语言先验、冗余层）能补位；这解释了论文-33 里"单层可跳、成组才崩"的现象——冗余不是均匀分布的，它集中在语言结构强的地方。

3. **为什么路径特征没用**：在大模型里，"收敛"不是内部状态的稳定，而是"语言层面已确定"——所以置信度（≈语言可预测性的读数）已经够用，内部路径的稳定性没有额外信息。

4. **判定器要看语言层**：与其看"内部算完了没有"，不如看"语言结构够不够强、够不够确定"——这是把"判定"从机制层移到语言层。

五、可检验推论

1. 语言结构强（低熵）的位置，早退掉点应显著小于高熵位置——可在更难的语料上复测。

2. 层冗余的分布应随语料变化：语言结构更强的领域（如程式化文本）冗余应更高。

3. 若判定器加入更强的语言结构特征（依存句法、上下文重复模式），应进一步接近 oracle。

4. 换更大模型：语言结构强度的作用应保持，且高熵位置的"需要更深"应更明显（因为大模型的深层能力更强）。

六、局限

第一，模型极小（GPT-2 small）、语料窄（tinyshakespeare），整体准确率仅 17%，不能外推到真实大模型。

第二，语言结构强度用"熵三分位"做代理，比较粗；词类/位置/重复特征也很粗。

第三，熵是按位置算的，同一位置的"语言结构"其实依赖上下文长度与质量，未做更细的控制。

第四，判定器训练标签是"预测是否正确"，与"退出后最终答案是否正确"不完全等价。

第五，我们没有测真正的语言结构标注（依存树、篇章结构），只用统计代理。

第六，这是对"语言层"的初步实验设计，结论是方向性的；我们对大模型的理解有限，不声称超过已有的early-exit / 语言模型分析研究。

七、与已有工作的位置

- early exit / adaptive computation：本实验的贡献是用"语言结构强度"分组，而不是按层或按 token 类别分组。

- 语言模型的内部分析（探针、熵、可预测性）：本实验把"可预测性"当作自变量，而不是当作解释变量。

- 与论文-33（层路径与冗余）的关系：论文-33 说明结构跨层展开且带冗余；本篇说明冗余集中分布在语言结构强的位置。

附：标注

- **[事实]** 三张表的数字来自本次 GPT-2 实验（数据文件：数据-语言层实验.json）。

- **[解释]** "语言结构强⇒早收敛/冗余高/可早退"——对数据的解读。

- **[推测]** 四条可检验推论——方向性，未验证。

- **[不声称]** 不声称适用于大模型；不声称路径特征永远无用（只在本次设置下没优势）；不声称"语言结构"已被严格定义。

> 本文由何淇创作，DeepSeek（AI）协助整理与写作；观点与责任属于作者。